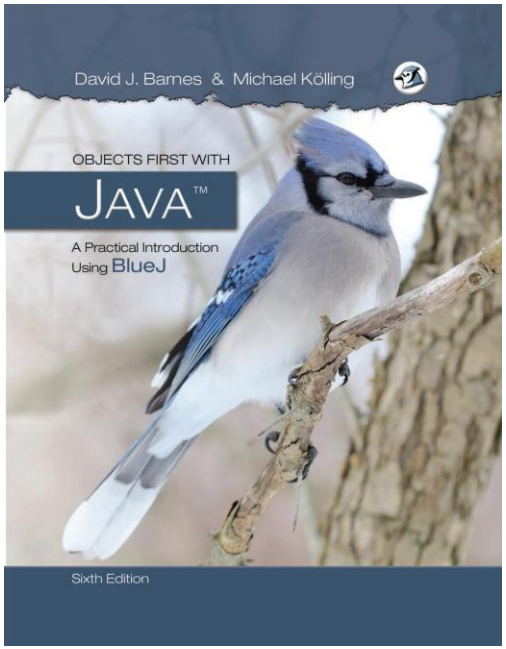
Unidad 6

Comportamientos más avanzados



Esta presentación es una traducción y adaptación de las presentaciones proporcionadas por la editorial Pearson de los capítulos 4, 6 y 7 del libro "Objects First with Java: A Practical Introduction Using BlueJ, 6th Edition" escrito por David J. Barnes y Michael Kolling" © 2017

Pearson

Principales conceptos

- Collections (*List*, *Set* y *Map*).
- Clases wrapper (envoltorio).
- Comportamiento aleatorio (*Random*)
- Variables y métodos de clase (static). Constantes (final)
- Tipo enumerado (*Enum*)
- Definición y uso del método ***main***

Principales conceptos

- Collections (*List*, *Set* y *Map*).
- Clases wrapper (envoltorio).
- Comportamiento aleatorio (*Random*)
- Variables y métodos de clase (static). Constantes (final)
- Tipo enumerado (*Enum*)
- Definición y uso del método ***main***

Colecciones (recordatorio)

- Agrupar objetos es un requisito recurrente.
  - El paquete **java.util** contiene múltiples clases para hacerlo llamadas **Colecciones**.
  - Son clases de la Biblioteca Java ”listas para ser usadas”.



El término **Colecciones** se puede considerar un término “paraguas” para agrupar las diferentes estructuras específicas disponibles en Java

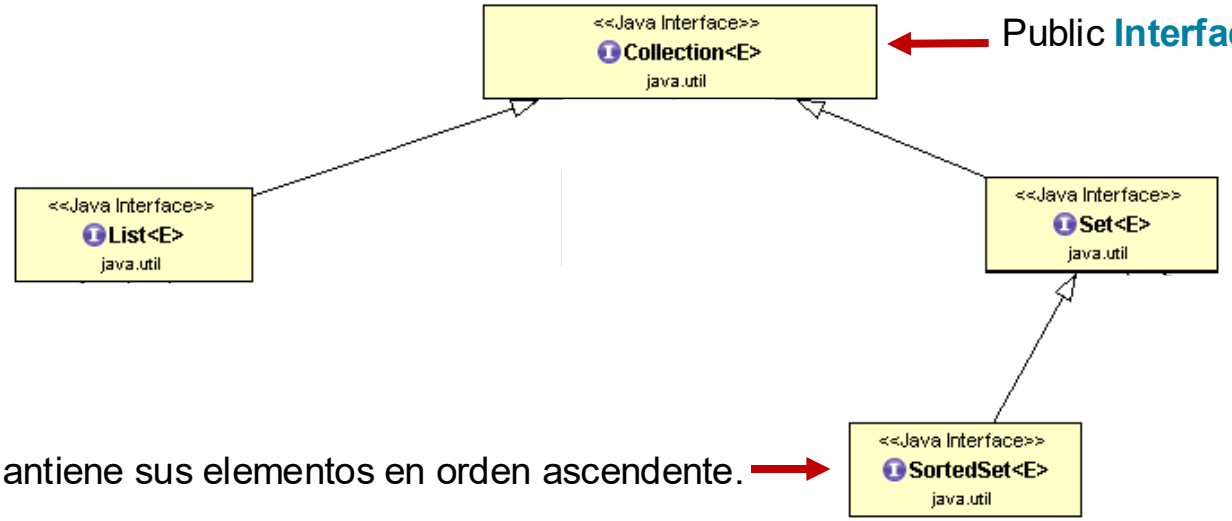
  - Pueden **contener elementos** (objetos)
  - Algunas pueden mantenerlos en un **cierto orden**
  - Ofrecen **acceso a los elementos**:
    - Eficientemente
    - Basándose en características específicas (posición, clave)
  - No representan nada por sí mismas, son meta-estructuras

Colecciones (recordatorio)

- Todas las **Colecciones** permiten:
  - Añadir elementos (*add*)
  - Eliminar elementos (*remove*)
  - Comprobar si se tiene un elemento (*contains*)
  - Recuperar la cantidad real de elementos que contiene (*size*)
  - Comprobar si están vacías (*isEmpty*)
- Son estructuras específicas adecuadas para casos de uso específicos

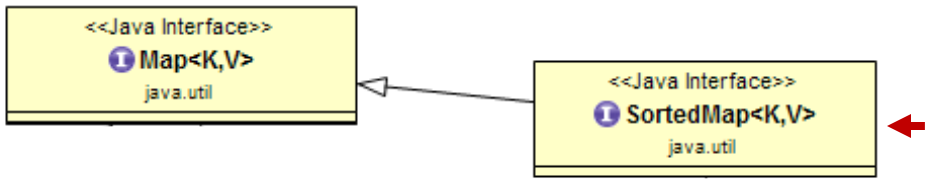
Tipos de colecciones

- Las colecciones se agrupan en 2 tipos llamados contenedores aunque realmente son *interfaces**:
  - **El interfaz Collection** propiamente dicho → Secuencia de elementos que cumple una serie de reglas (ordenada o no, con o sin elementos duplicados, etc. …)



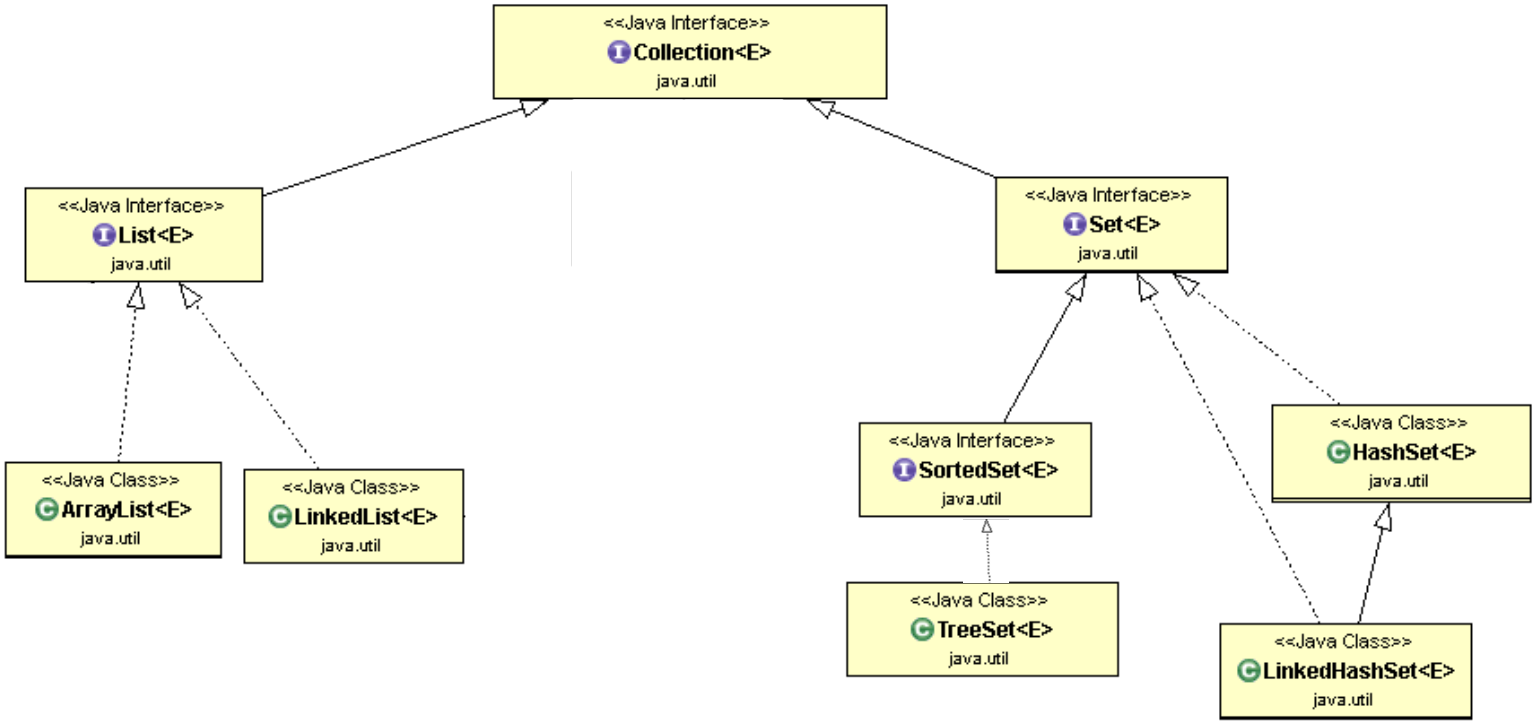
Tipos de colecciones

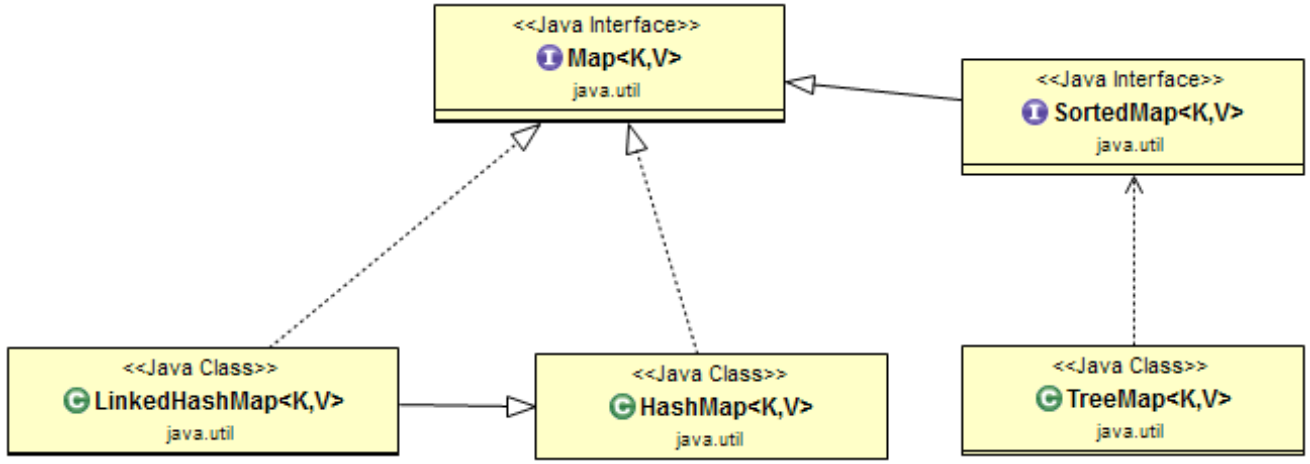
- Las colecciones se agrupan en 2 tipos llamados contenedores aunque realmente son *interfaces**:
  - **El interfaz Map** → Par *key-value* (asociativo).
    - Permite acceso y búqueda de elementos usando la clave (key).
    - Los nombres de los métodos difieren con respecto a los de los tipos Collection



Map que mantiene sus mapeos en orden ascendiente por clave

¿Quién implementa esas *interfaces*?





Tipos de colecciones polimórficas

- Distintas clases colección ofrecen interfaces similares; ej.:
  - `ArrayList` y `LinkedList`
  - `HashSet` y `TreeSet`
- Existen tipos que unifican esas similitudes:
  - `List` - `Set`
- El **Polimorfismo (polymorphism)** nos permite ignorar los tipos más específicos en la mayoría de los casos.
- Creamos objectos de un tipo específico, pero ...
- … las variables se declaran del tipo más general:

```
List<Track> tracks = new LinkedList<>();

Map<String, String> responseMap = new HashMap<>();
```

Clases parametrizadas (genéricas)

```
private List<String> responses;
```

- Al leer la documentación official se ve un **parámetro de tipo**:
  - ArrayList <**E**>
- Estos nombres de tipo reaparecen en los parámetros y tipos de retorno de los métodos de clases:
  - `E get(int index)`
  - `boolean add(E e)`
- Los tipos en la documentación son espacios para los tipos utilizados en la práctica:
- Un **ArrayList**<TicketMachine> tiene métodos como:
  - `TicketMachine get (int index)`
  - `boolean add (TicketMachine tm)`

List, Set y Map

- Son formas alternativas de agrupar objetos.
- Implementaciones disponibles:
  - `List`: `ArrayList`, `LinkedList`
  - `Set`: `TreeSet` , `HashSet`, `LinkedHashSet`
  - `Map`: `TreeMap`, `HashMap`, `LinkedHashMap`
- Un `HashMap` **no** está relacionado con un `HashSet`, a pesar de que sus nombres son similares.
  - `HashSet` es más cercano a un `ArrayList`.
- Los nombres tienen 2 partes: "Array" "List":
  - Inicio: cómo se implementa.
  - Final: forma en la que se organiza(tipo de colección (List, Map, Set)).

A continuación veremos con más detalle estas estructuras…

List

Imagina el siguiente escenario:

Estamos en un soporte técnico informático.

Procesamos todos los tickets que llegan **en orden de llegada**.

Estos tickets necesitan procesarse rápido

REQUISITOS:

- Añadir elementos al **final** de una colección ordenada
- Acceder a ellos desde el **inicio**
- A veces, colocar algunos elementos en **medio**

- **¿SOLUCIÓN?**

Imagina el siguiente escenario:

Estamos en un soporte técnico informático.

Procesamos todos los tickets que llegan **en orden de llegada**.

Estos tickets necesitan procesarse rápido

REQUISITOS:

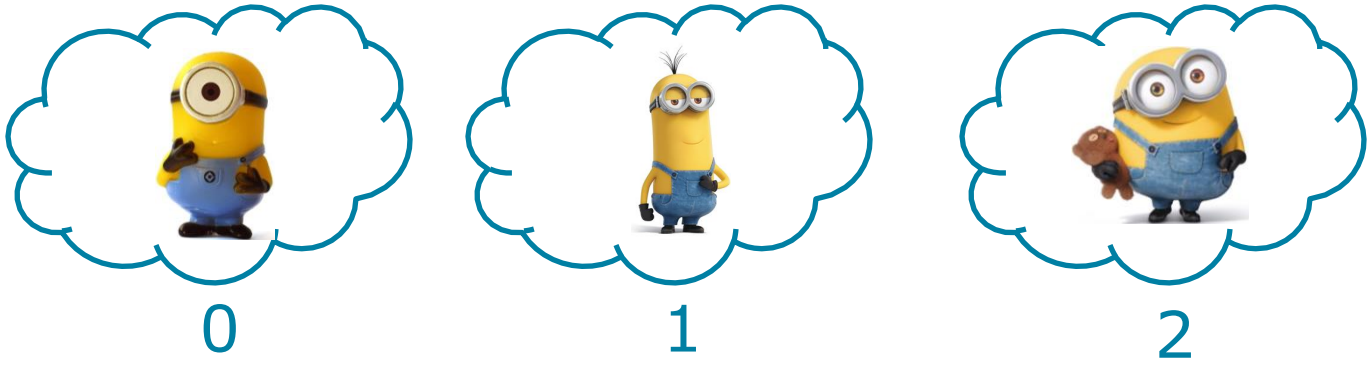
- Añadir elementos al **final** de una colección ordenada
- Acceder a ellos desde el **inicio**
- A veces, colocar algunos elementos en **medio**

– **¿SOLUCIÓN?**

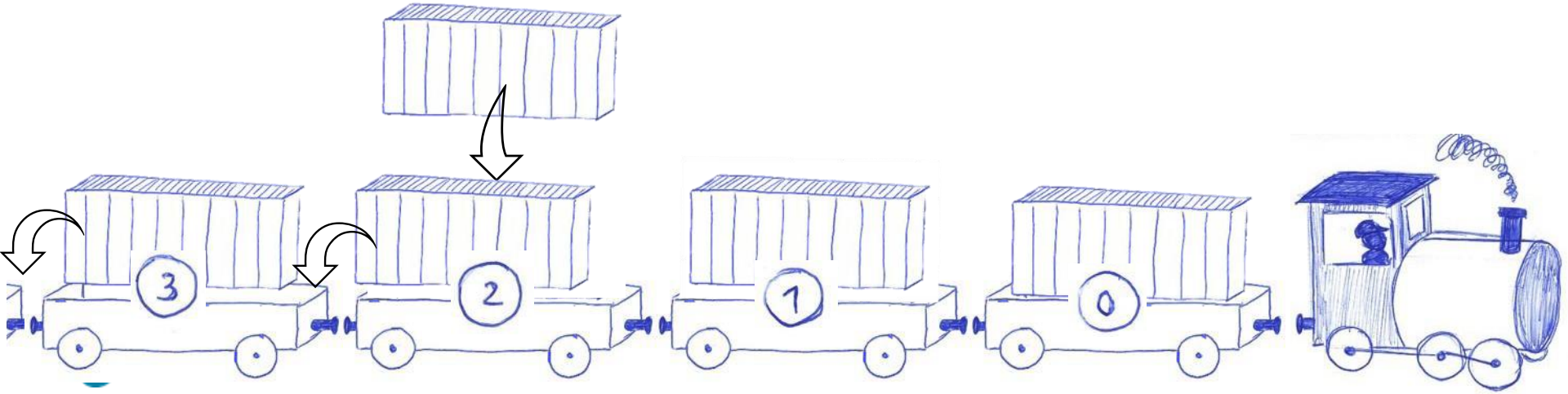
```
Usar una Lista (List)
```

List

- Cada elemento tiene una posición

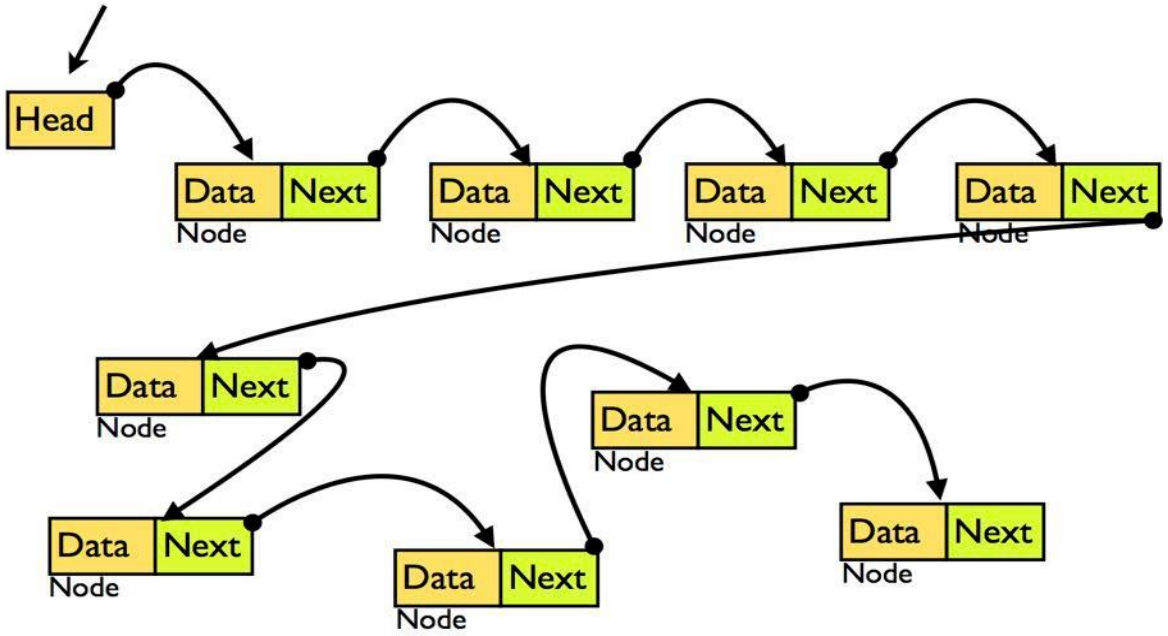


- Propiedades de una lista:
  - los elementos pueden añadirse en cualquier posición
  - podemos recorrerla



List

- Idea: **Nodos** encadenados
- Cada nodo tiene una **referencia al siguiente nodo** (Next) y contiene un **objecto de datos** (Data)
- Inconveniente: no permite acceso directo
  - tenemos que iterar sobre todos las posiciones predecesoras hasta llegar a un objeto concreto



List

¡¡Explicado en Unidad 03!!

ArrayList: Colección secuencial de elementos implementada como un array.

- Acceso a posición en tiempo constante → Muy eficiente.
- Añadir o eliminar elementos por el medio → Menos eficiente (aunque se beneficia de *System.arrayCopy()* para mover varios elementos a la vez).
- puede tener objectos duplicados

Datos ordenados (con método *sort* de *Collections* utilizando *Comparator*).

List

LinkedList: Colección secuencial de elementos implementada como una lista doblemente enlazada.

- Acceso a posición en tiempo lineal → Menos eficiente.
- Añadir elementos en cualquier parte de la lista o eliminar elementos mediante el iterador → Muy eficiente
- Puede tener objectos duplicados
- Tiene algunos métodos adicionales que permiten su uso tanto como una pila (*stack*) como una cola (*queue*):
  - **addFirst()** → Añade al principio de la lista
  - **remove()**, **removeFirst()** → Elimina y devuelve el primer elemento
  - **poll()** → Igual a *remove()* pero devuelve null si la lista está vacía
  - **offer()**, **add()**, **addLast()** → Al final de la lista
  - **removeLast()** → Elimina y devuelve el ultimo elemento

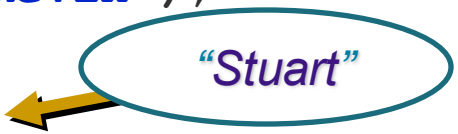
Datos ordenados (con método *sort* de *Collections* utilizando *Comparator*).

List

LinkedList:

Ejemplo:

```
List<String> list   =   new   LinkedList<>();
list.add("Stuart");
list.add("Kevin");
```




```
String primero = list.removeFirst();

String nuevoPrimero = list.getFirst();
```

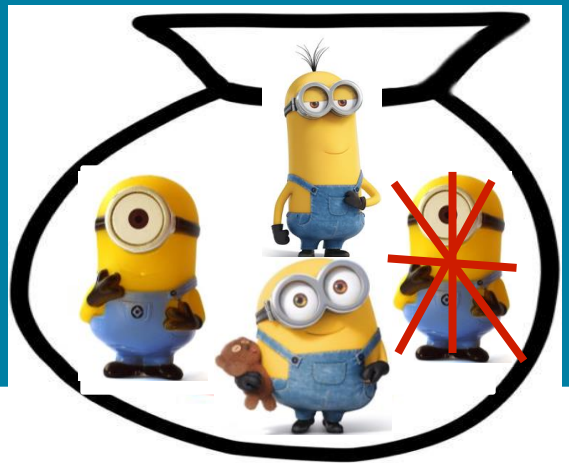


Set

Imagina el siguiente escenario:

Vamos a hacer un sorteo. Cada participante sólo puede entrar en el sorteo **una vez**.

Hay participantes que van a intentar hacer trampa, pero sólo permitimos **una entrada por persona**.



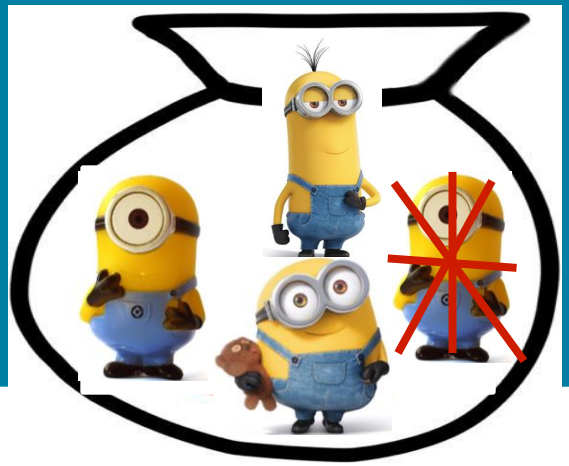
REQUISITOS:

- Queremos:
  - Cada elemento exista **sólo una vez**, no hay duplicados
  - Añadir elementos en nuestra colección (**ordenada** o **no ordenada)**
  - Comprobar si una entrada **ya existe**
- **¿SOLUCIÓN?**

Imagina el siguiente escenario:

Vamos a hacer un sorteo. Cada participante sólo puede entrar en el sorteo **una vez**.

Hay participantes que van a intentar hacer trampa, pero sólo permitimos **una entrada por persona**.

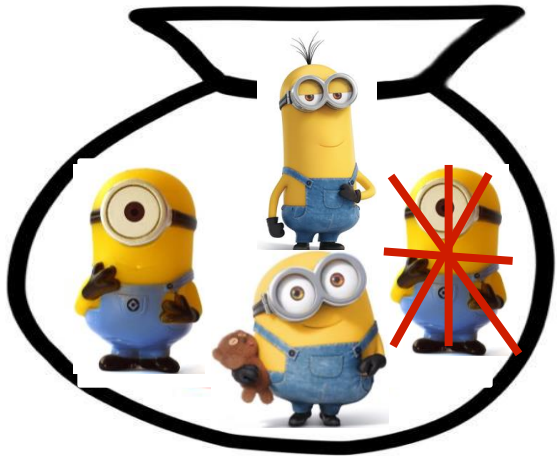


REQUISITOS:

- Queremos:
  - Cada elemento exista **sólo una vez**, no hay duplicados
  - Añadir elementos en nuestra colección (**ordenada** o **no ordenada)**
  - Comprobar si una entrada **ya existe**
- **¿SOLUCIÓN?**

Usar un **Conjunto** (**Set**)

Sets



- Colección de elementos sin duplicados

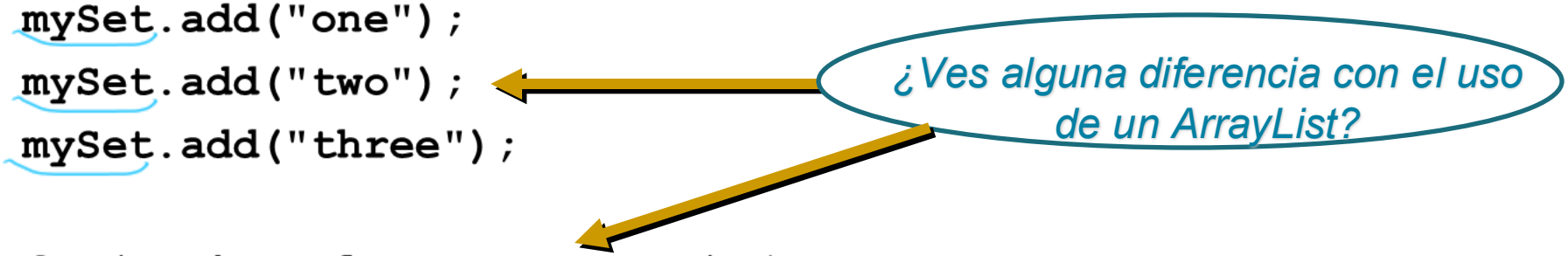
- **TreeSet: conjunto ordenado**
  - según el orden establecido por *compareTo* o *Comparator*.
  - Accesos y búsquedas realmente eficientes.
- **HashSet: conjunto sin orden concreto**
  - Usa técnica de *hash* para almacenar y localizar los elementos. ¡MUY RÁPIDA Y EFICIENTE!
- **LinkedHashSet: conjunto ordenado por orden de inserción**
  - HashSet con una implementación de lista enlazada

Sets

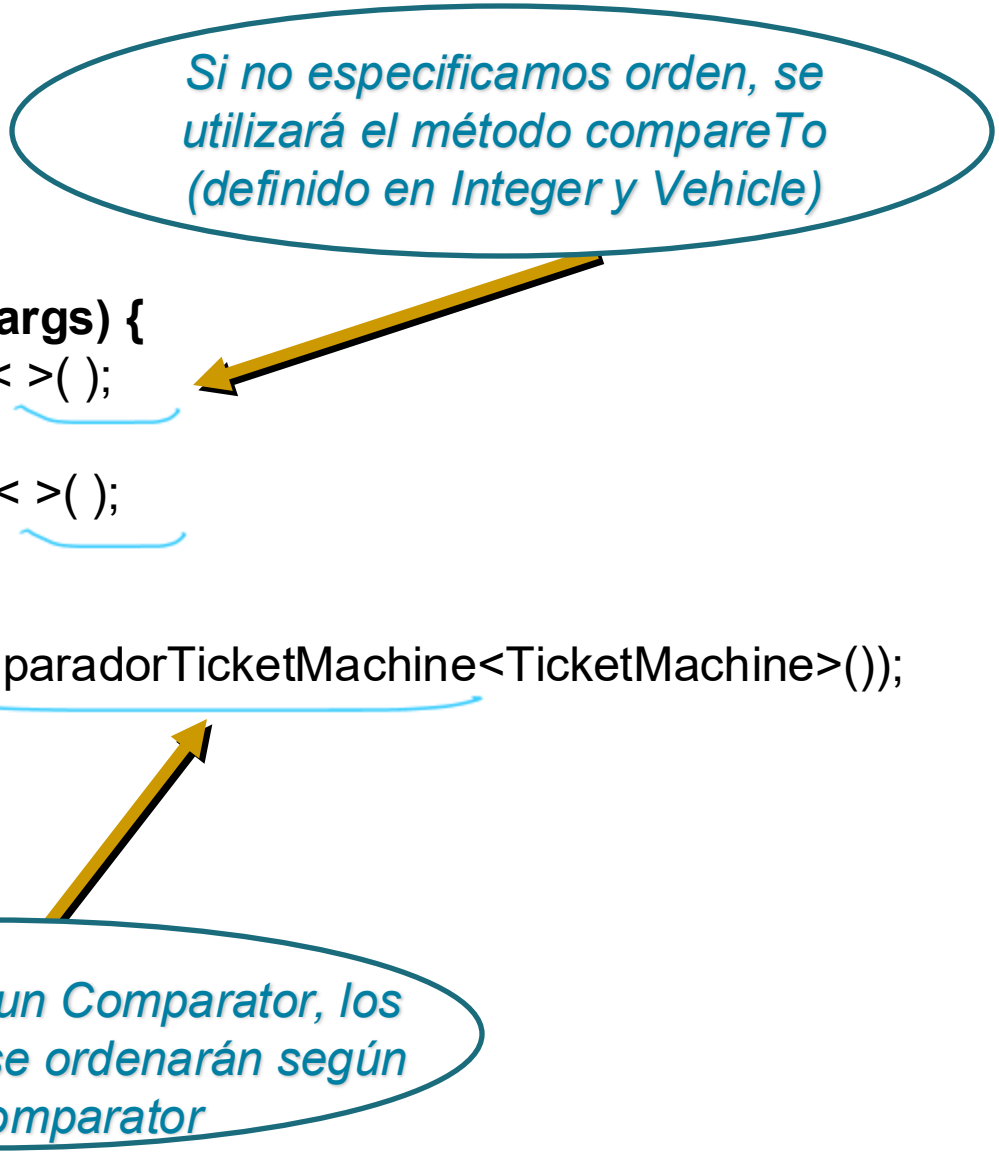
```
import java.util.HashSet;
...
HashSet<String> mySet = new HashSet<>();

mySet.add("one");
mySet.add("two");
mySet.add("three");

for(String element : mySet) {
    do something with element
}
```



Sets


Si no especificamos orden, se utilizará el método compareTo (definido en Integer y Vehicle)
public class PruebaSet {
public static void main(String[] args) {
Set<Integer> c1 = new TreeSet< >( );
Set<Vehicle> c2 = new TreeSet< >( );
Set<TicketMachine> c3 =
new TreeSet< >(new ComparadorTicketMachine<TicketMachine>());
}
}
Si especificamos un Comparator, los elementos de c3 se ordenarán según dicho Comparator

Map

Imagina el siguiente escenario:

Estamos en un taller de vehículos. Tenemos diferentes clientes y queremos reparar sus vehículos.

Cada cliente quiere **su** vehículo reparado.

REQUISITOS:

- Queremos:
  - Añadir elementos en nuestra colección
  - Buscar y recuperar vehículos por cliente
    - por ejemplo por su nombre
- **¿SOLUCIÓN?**

Imagina el siguiente escenario:

Estamos en un taller de vehículos. Tenemos diferentes clientes y queremos reparar sus vehículos.

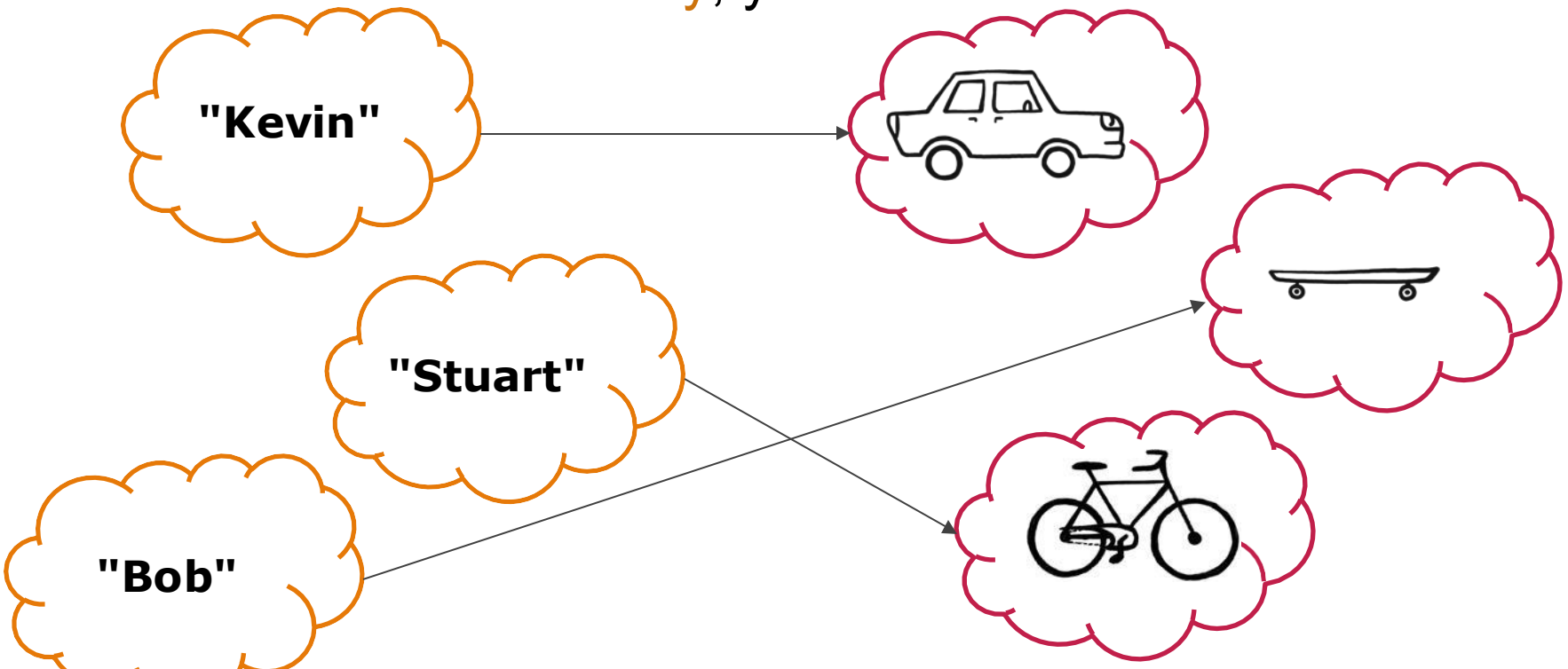
Cada cliente quiere **su** vehículo reparado.

REQUISITOS:

- Queremos:
  - Añadir elementos en nuestra colección
  - Buscar y recuperar vehículos por cliente
    - por ejemplo por su nombre
- **¿SOLUCIÓN?** `Usar un Mapa (Map)`

Maps

- Colecciones de tamaño flexible que contiene **pares** de objetos.
- Parametrizados con **dos** tipos.
- Un par se forma de una **clave (key)** y un **valor (value)**.
  - key se usa para acceder fácilmente a value.
- Para buscar se envía una key, y se devuelve un value.



Maps

- Ejemplo: una guía telefónica
- Mapa con Strings como claves y valores



Maps

HashMap. Colección de pares clave-valor sin orden determinado.

- Acceso y búsquedas muy eficientes.

LinkedHashMap. Colección de pares clave-valor con objetos ordenados según el orden de inserción.

- Casi igual de eficiente que HashMap

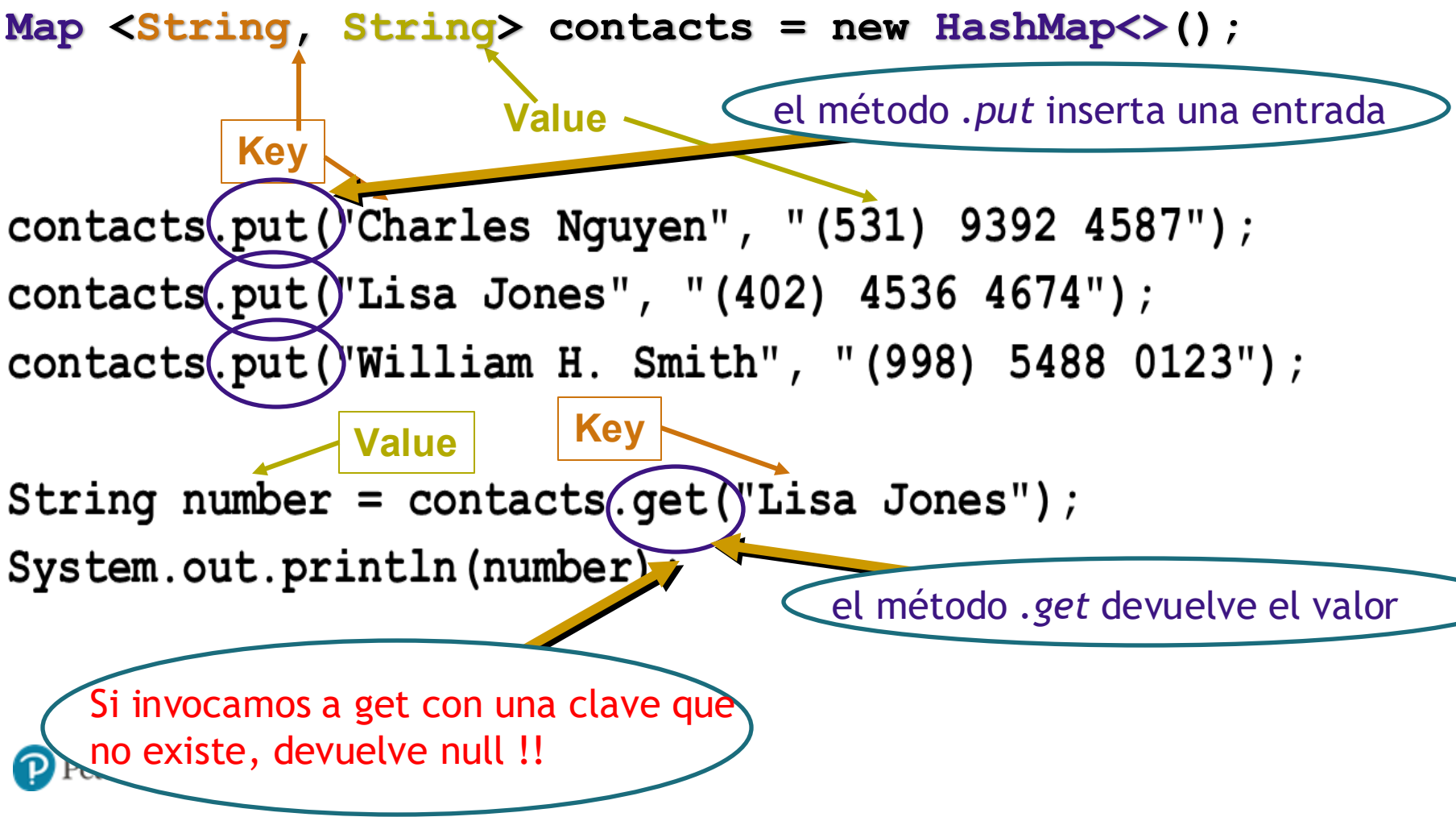
TreeMap. Colección de pares clave-valor ordenados por la clave (ascendente).

- El orden se establece por compareTo o Comparator

http://docs.oracle.com/javase/7/docs/api/java/util/Map.html

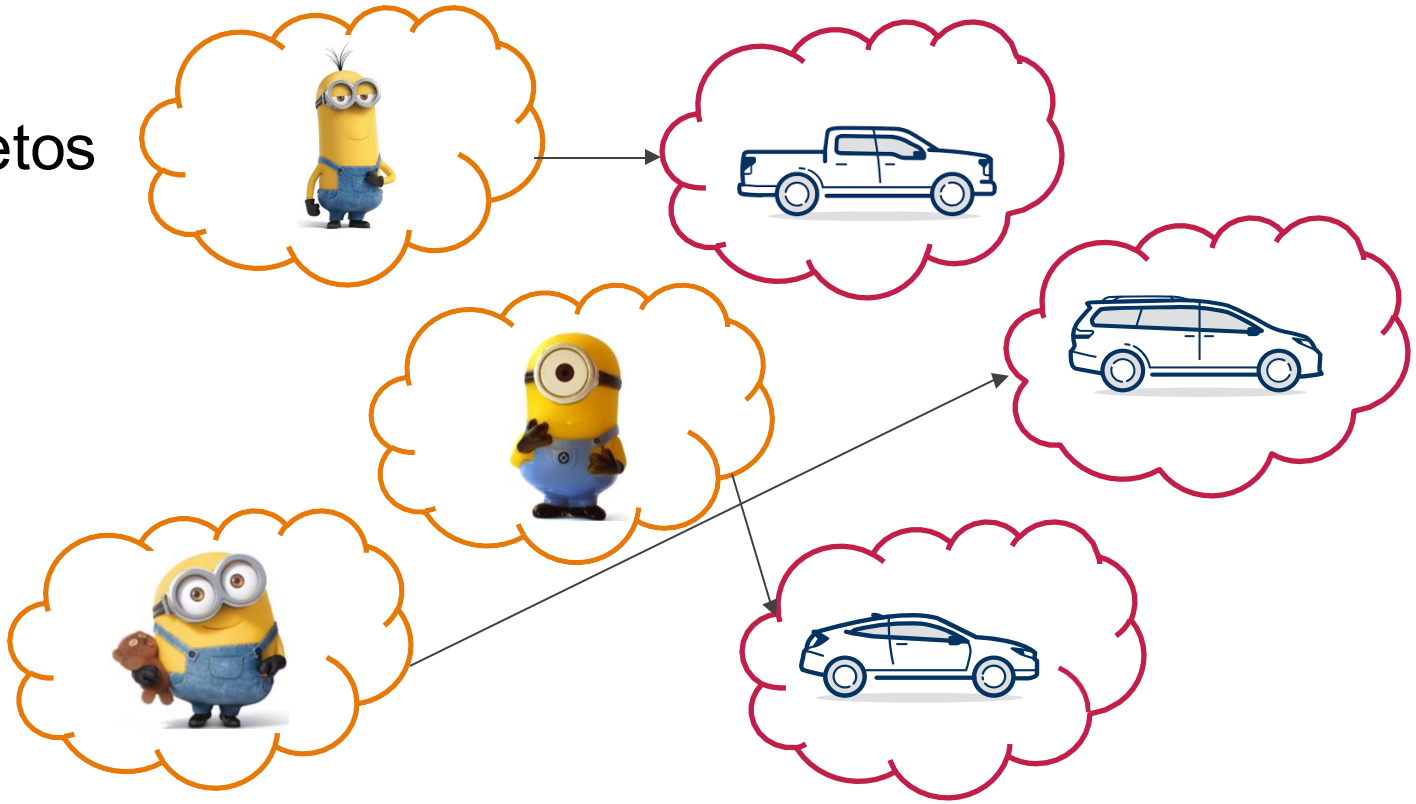
Maps

- **HashMap:** Colección de pares *key-value* sin un orden concreto cuya implementación se basa en tabla hash



Maps

- Normalmente, las claves no son numéricas:
  - Strings
  - otros objetos



```
workshop.put(kevin,truck);
workshop.put(stuart,coupe);
workshop.put(bob,van);
workshop.get(stuart);
```



Maps

```
public static void main(String[] args) {
  Map <Integer,String> mapa = new TreeMap< >();
  for (int i=0; i<4; i++){
        mapa.put(i, "Cadena" + i);
  }
  System.out.println("Mapa:" + mapa);
  System.out.println("1:" + mapa.get(1));
  mapa.remove(1);
  if (mapa.containsKey(3))
        System.out.println("Clave 3:" + "Encontrado");

  if (mapa.containsValue("Cadena3")) // Búsqueda de valor
        System.out.println("Valor Cadena3:" + "Encontrado");

  Set<Integer> claves = mapa.keySet(); //Obtener conjunto de claves
  System.out.println("Claves:" + claves);
  Collection<String> valores = mapa.values(); //Obtener colección valores
  System.out.println("Valores:" + valores);

  //Obtener un conjunto de pares
  Set<Map.Entry<Integer, String>> pares = mapa.entrySet();
  for (Map.Entry< Integer, String > p : pares){ //Recorrer los pares
        System.out.print("C:" + p.getKey() + ":" + "V:" + p.getValue() + " - ");
  }
}
```

Pearson

Maps

```
public static void main(String[] args) {
  Map <Integer,String> mapa = new TreeMap< >();
  for (int i=0; i<4; i++){
        mapa.put(i, "Cadena" + i);
  }
  System.out.println("Mapa:" + mapa);
  System.out.println("1:" + mapa.get(1));
  mapa.remove(1);
  if (mapa.containsKey(3))
        System.out.println("Clave 3:" + "Encontrado");

  if (mapa.containsValue("Cadena3")) // Búsqueda de valor
        System.out.println("Valor Cadena3:" + "Encontrado");

  Set<Integer> claves = mapa.keySet(); //Obtener conjunto de claves
  System.out.println("Claves:" + claves);
  Collection<String> valores = mapa.values(); //Obtener colección valores
  System.out.println("Valores:" + valores);

  //Obtener un conjunto de pares
  Set<Map.Entry<Integer, String>> pares = mapa.entrySet();
  for (Map.Entry< Integer, String > p : pares){ //Recorrer los pares
        System.out.print("C:" + p.getKey() + ":" + "V:" + p.getValue() + " - ");
  }
}
```

```
Mapa:{0=Cadena0, 1=Cadena1, 2=Cadena2, 3=Cadena3}
```

Maps

```
public static void main(String[] args) {
  Map <Integer,String> mapa = new TreeMap< >();
  for (int i=0; i<4; i++){
        mapa.put(i, "Cadena" + i);
  }
  System.out.println("Mapa:" + mapa);
  System.out.println("1:" + mapa.get(1));
  mapa.remove(1);
  if (mapa.containsKey(3))
        System.out.println("Clave 3:" + "Encontrado");

  if (mapa.containsValue("Cadena3")) // Búsqueda de valor
        System.out.println("Valor Cadena3:" + "Encontrado");

  Set<Integer> claves = mapa.keySet(); //Obtener conjunto de claves
  System.out.println("Claves:" + claves);
  Collection<String> valores = mapa.values(); //Obtener colección valores
  System.out.println("Valores:" + valores);

  //Obtener un conjunto de pares
  Set<Map.Entry<Integer, String>> pares = mapa.entrySet();
  for (Map.Entry< Integer, String > p : pares){ //Recorrer los pares
        System.out.print("C:" + p.getKey() + ":" + "V:" + p.getValue() + " - ");
  }
}
```

```
Mapa:{0=Cadena0, 1=Cadena1, 2=Cadena2, 3=Cadena3}
1:Cadena1
```

Maps

```
public static void main(String[] args) {
  Map <Integer,String> mapa = new TreeMap< >();
  for (int i=0; i<4; i++){
        mapa.put(i, "Cadena" + i);
  }
  System.out.println("Mapa:" + mapa);
  System.out.println("1:" + mapa.get(1));
  mapa.remove(1);
  if (mapa.containsKey(3))
        System.out.println("Clave 3:" + "Encontrado");

  if (mapa.containsValue("Cadena3")) // Búsqueda de valor
        System.out.println("Valor Cadena3:" + "Encontrado");

  Set<Integer> claves = mapa.keySet(); //Obtener conjunto de claves
  System.out.println("Claves:" + claves);
  Collection<String> valores = mapa.values(); //Obtener colección valores
  System.out.println("Valores:" + valores);

  //Obtener un conjunto de pares
  Set<Map.Entry<Integer, String>> pares = mapa.entrySet();
  for (Map.Entry< Integer, String > p : pares){ //Recorrer los pares
        System.out.print("C:" + p.getKey() + ":" + "V:" + p.getValue() + " - ");
  }
}
```

```
Mapa:{0=Cadena0, 1=Cadena1, 2=Cadena2, 3=Cadena3}
1:Cadena1
```

Maps

```
public static void main(String[] args) {
  Map <Integer,String> mapa = new TreeMap< >();
  for (int i=0; i<4; i++){
        mapa.put(i, "Cadena" + i);
  }
  System.out.println("Mapa:" + mapa);
  System.out.println("1:" + mapa.get(1));
  mapa.remove(1);
  if (mapa.containsKey(3))
        System.out.println("Clave 3:" + "Encontrado");

  if (mapa.containsValue("Cadena3")) // Búsqueda de valor
        System.out.println("Valor Cadena3:" + "Encontrado");

  Set<Integer> claves = mapa.keySet(); //Obtener conjunto de claves
  System.out.println("Claves:" + claves);
  Collection<String> valores = mapa.values(); //Obtener colección valores
  System.out.println("Valores:" + valores);

  //Obtener un conjunto de pares
  Set<Map.Entry<Integer, String>> pares = mapa.entrySet();
  for (Map.Entry< Integer, String > p : pares){ //Recorrer los pares
        System.out.print("C:" + p.getKey() + ":" + "V:" + p.getValue() + " - ");
  }
}
```

```
Mapa:{0=Cadena0, 1=Cadena1, 2=Cadena2, 3=Cadena3}
1:Cadena1
Clave 3:Encontrado
```

Maps

```
public static void main(String[] args) {
  Map <Integer,String> mapa = new TreeMap< >();
  for (int i=0; i<4; i++){
        mapa.put(i, "Cadena" + i);
  }
  System.out.println("Mapa:" + mapa);
  System.out.println("1:" + mapa.get(1));
  mapa.remove(1);
  if (mapa.containsKey(3))
        System.out.println("Clave 3:" + "Encontrado");

  if (mapa.containsValue("Cadena3")) // Búsqueda de valor
        System.out.println("Valor Cadena3:" + "Encontrado");

  Set<Integer> claves = mapa.keySet(); //Obtener conjunto de claves
  System.out.println("Claves:" + claves);
  Collection<String> valores = mapa.values(); //Obtener colección valores
  System.out.println("Valores:" + valores);

  //Obtener un conjunto de pares
  Set<Map.Entry<Integer, String>> pares = mapa.entrySet();
  for (Map.Entry< Integer, String > p : pares){ //Recorrer los pares
        System.out.print("C:" + p.getKey() + ":" + "V:" + p.getValue() + " - ");
  }
}
```

```
Mapa:{0=Cadena0, 1=Cadena1, 2=Cadena2, 3=Cadena3}
1:Cadena1
Clave 3:Encontrado
Valor Cadena3:Encontrado
```

Maps

```
public static void main(String[] args) {
  Map <Integer,String> mapa = new TreeMap< >();
  for (int i=0; i<4; i++){
        mapa.put(i, "Cadena" + i);
  }
  System.out.println("Mapa:" + mapa);
  System.out.println("1:" + mapa.get(1));
  mapa.remove(1);
  if (mapa.containsKey(3))
        System.out.println("Clave 3:" + "Encontrado");

  if (mapa.containsValue("Cadena3")) // Búsqueda de valor
        System.out.println("Valor Cadena3:" + "Encontrado");

  Set<Integer> claves = mapa.keySet(); //Obtener conjunto de claves
  System.out.println("Claves:" + claves);
  Collection<String> valores = mapa.values(); //Obtener colección valores
  System.out.println("Valores:" + valores);

  //Obtener un conjunto de pares
  Set<Map.Entry<Integer, String>> pares = mapa.entrySet();
  for (Map.Entry< Integer, String > p : pares){ //Recorrer los pares
        System.out.print("C:" + p.getKey() + ":" + "V:" + p.getValue() + " - ");
  }
}
```

```
Mapa:{0=Cadena0, 1=Cadena1, 2=Cadena2, 3=Cadena3}
1:Cadena1
Clave 3:Encontrado
Valor Cadena3:Encontrado
Claves:[0, 2, 3]
```

Maps

```
public static void main(String[] args) {
  Map <Integer,String> mapa = new TreeMap< >();
  for (int i=0; i<4; i++){
        mapa.put(i, "Cadena" + i);
  }
  System.out.println("Mapa:" + mapa);
  System.out.println("1:" + mapa.get(1));
  mapa.remove(1);
  if (mapa.containsKey(3))
        System.out.println("Clave 3:" + "Encontrado");

  if (mapa.containsValue("Cadena3")) // Búsqueda de valor
        System.out.println("Valor Cadena3:" + "Encontrado");

  Set<Integer> claves = mapa.keySet(); //Obtener conjunto de claves
  System.out.println("Claves:" + claves);
  Collection<String> valores = mapa.values(); //Obtener colección valores
  System.out.println("Valores:" + valores);

  //Obtener un conjunto de pares
  Set<Map.Entry<Integer, String>> pares = mapa.entrySet();
  for (Map.Entry< Integer, String > p : pares){ //Recorrer los pares
        System.out.print("C:" + p.getKey() + ":" + "V:" + p.getValue() + " - ");
  }
}
```

```
Mapa:{0=Cadena0, 1=Cadena1, 2=Cadena2, 3=Cadena3}
1:Cadena1
Clave 3:Encontrado
Valor Cadena3:Encontrado
Claves:[0, 2, 3]
Valores:[Cadena0, Cadena2, Cadena3]
```

Maps

```
public static void main(String[] args) {
  Map <Integer,String> mapa = new TreeMap< >();
  for (int i=0; i<4; i++){
        mapa.put(i, "Cadena" + i);
  }
  System.out.println("Mapa:" + mapa);
  System.out.println("1:" + mapa.get(1));
  mapa.remove(1);
  if (mapa.containsKey(3))
        System.out.println("Clave 3:" + "Encontrado");

  if (mapa.containsValue("Cadena3")) // Búsqueda de valor
        System.out.println("Valor Cadena3:" + "Encontrado");

  Set<Integer> claves = mapa.keySet(); //Obtener conjunto de claves
  System.out.println("Claves:" + claves);
  Collection<String> valores = mapa.values(); //Obtener colección valores
  System.out.println("Valores:" + valores);

  //Obtener un conjunto de pares
  Set<Map.Entry<Integer, String>> pares = mapa.entrySet();
  for (Map.Entry< Integer, String > p : pares){ //Recorrer los pares
        System.out.print("C:" + p.getKey() + ":" + "V:" + p.getValue() + " - ");
  }
}
```

```
Mapa:{0=Cadena0, 1=Cadena1, 2=Cadena2, 3=Cadena3}
1:Cadena1
Clave 3:Encontrado
Valor Cadena3:Encontrado
Claves:[0, 2, 3]
Valores:[Cadena0, Cadena2, Cadena3]
C:0:V:Cadena0 - C:2:V:Cadena2 - C:3:V:Cadena3 -
```

Maps

Mapa de vehículos (Marca-Modelo) como “values” y sus matrículas como “keys”

```
public static void main(String[] args) {
  Map <String,Vehicle> mapa = new TreeMap< >();
  mapa.put(“111A”, new Vehicle(”Opel”,”Crossland"));
  mapa.put(“333C”, new Vehicle(“Peugeot”,”3008"));
  mapa.put(“222B”, new Vehicle(“Citroen”,”C3"));
   System.out.println(mapa);

 Vehicle v = mapa.get(“111A”));
   System.out.println(“Obtenido vehículo: “+v));
   mapa.remove(“222B”);
   if (mapa.containsKey(“333C”))
         System.out.println("Encontrado");
   if (mapa.containsValue(v)) // Búsqueda de valor
          System.out.println("Encontrado");

    Set<String> claves = mapa.keySet(); //Obtener conjunto de claves
    System.out.println(claves);
    Collection<Vehicle> valores = mapa.values(); //Obtener colección valores
    System.out.println(valores);

   //Obtener un conjunto de pares
   Set<Map.Entry<String, Vehicle>> pares = mapa.entrySet();
   for (Map.Entry<String, Vehicle> p : pares){ //Recorrer los pares
         System.out.print(p.getKey() + “>>" + p.getValue() + “<< - ");
   }
}
```

Pearson

Maps

Mapa de vehículos (Marca-Modelo) como “values” y sus matrículas como “keys”

```
public static void main(String[] args) {
  Map <String,Vehicle> mapa = new TreeMap< >();
  mapa.put(“111A”, new Vehicle(”Opel”,”Crossland"));
  mapa.put(“333C”, new Vehicle(“Peugeot”,”3008"));
  mapa.put(“222B”, new Vehicle(“Citroen”,”C3"));
   System.out.println(mapa);

 Vehicle v = mapa.get(“111A”));
   System.out.println(“Obtenido vehículo: “+v));
   mapa.remove(“222B”);
   if (mapa.containsKey(“333C”))
         System.out.println("Encontrado");
   if (mapa.containsValue(v)) // Búsqueda de valor
          System.out.println("Encontrado");

    Set<String> claves = mapa.keySet(); //Obtener conjunto de claves
    System.out.println(claves);
    Collection<Vehicle> valores = mapa.values(); //Obtener colección valores
    System.out.println(valores);

   //Obtener un conjunto de pares
   Set<Map.Entry<String, Vehicle>> pares = mapa.entrySet();
   for (Map.Entry<String, Vehicle> p : pares){ //Recorrer los pares
         System.out.print(p.getKey() + “>>" + p.getValue() + “<< - ");
   }
}
```

```
{111A=Opel-Crossland, 222B=Citroen-C3, 333C=Peugeot-3008}
```

Maps

Mapa de vehículos (Marca-Modelo) como “values” y sus matrículas como “keys”

```
public static void main(String[] args) {
  Map <String,Vehicle> mapa = new TreeMap< >();
  mapa.put(“111A”, new Vehicle(”Opel”,”Crossland"));
  mapa.put(“333C”, new Vehicle(“Peugeot”,”3008"));
  mapa.put(“222B”, new Vehicle(“Citroen”,”C3"));
   System.out.println(mapa);

 Vehicle v = mapa.get(“111A”));
   System.out.println(“Obtenido vehículo: “+v));
   mapa.remove(“222B”);
   if (mapa.containsKey(“333C”))
         System.out.println("Encontrado");
   if (mapa.containsValue(v)) // Búsqueda de valor
          System.out.println("Encontrado");

    Set<String> claves = mapa.keySet(); //Obtener conjunto de claves
    System.out.println(claves);
    Collection<Vehicle> valores = mapa.values(); //Obtener colección valores
    System.out.println(valores);

   //Obtener un conjunto de pares
   Set<Map.Entry<String, Vehicle>> pares = mapa.entrySet();
   for (Map.Entry<String, Vehicle> p : pares){ //Recorrer los pares
         System.out.print(p.getKey() + “>>" + p.getValue() + “<< - ");
   }
}
```

```
{111A=Opel-Crossland, 222B=Citroen-C3, 333C=Peugeot-3008}
Obtenido vehículo: Opel-Crossland
```

Maps

Mapa de vehículos (Marca-Modelo) como “values” y sus matrículas como “keys”

```
public static void main(String[] args) {
  Map <String,Vehicle> mapa = new TreeMap< >();
  mapa.put(“111A”, new Vehicle(”Opel”,”Crossland"));
  mapa.put(“333C”, new Vehicle(“Peugeot”,”3008"));
  mapa.put(“222B”, new Vehicle(“Citroen”,”C3"));
   System.out.println(mapa);

 Vehicle v = mapa.get(“111A”));
   System.out.println(“Obtenido vehículo: “+v));
   mapa.remove(“222B”);
   if (mapa.containsKey(“333C”))
         System.out.println("Encontrado");
   if (mapa.containsValue(v)) // Búsqueda de valor
          System.out.println("Encontrado");

    Set<String> claves = mapa.keySet(); //Obtener conjunto de claves
    System.out.println(claves);
    Collection<Vehicle> valores = mapa.values(); //Obtener colección valores
    System.out.println(valores);

   //Obtener un conjunto de pares
   Set<Map.Entry<String, Vehicle>> pares = mapa.entrySet();
   for (Map.Entry<String, Vehicle> p : pares){ //Recorrer los pares
         System.out.print(p.getKey() + “>>" + p.getValue() + “<< - ");
   }
}
```

```
{111A=Opel-Crossland, 222B=Citroen-C3, 333C=Peugeot-3008}
Obtenido vehículo: Opel-Crossland
```

Maps

Mapa de vehículos (Marca-Modelo) como “values” y sus matrículas como “keys”

```
public static void main(String[] args) {
  Map <String,Vehicle> mapa = new TreeMap< >();
  mapa.put(“111A”, new Vehicle(”Opel”,”Crossland"));
  mapa.put(“333C”, new Vehicle(“Peugeot”,”3008"));
  mapa.put(“222B”, new Vehicle(“Citroen”,”C3"));
   System.out.println(mapa);

 Vehicle v = mapa.get(“111A”));
   System.out.println(“Obtenido vehículo: “+v));
   mapa.remove(“222B”);
   if (mapa.containsKey(“333C”))
         System.out.println("Encontrado");
   if (mapa.containsValue(v)) // Búsqueda de valor
          System.out.println("Encontrado");

    Set<String> claves = mapa.keySet(); //Obtener conjunto de claves
    System.out.println(claves);
    Collection<Vehicle> valores = mapa.values(); //Obtener colección valores
    System.out.println(valores);

   //Obtener un conjunto de pares
   Set<Map.Entry<String, Vehicle>> pares = mapa.entrySet();
   for (Map.Entry<String, Vehicle> p : pares){ //Recorrer los pares
         System.out.print(p.getKey() + “>>" + p.getValue() + “<< - ");
   }
}
```

```
{111A=Opel-Crossland, 222B=Citroen-C3, 333C=Peugeot-3008}
Obtenido vehículo: Opel-Crossland
Encontrado
```

Maps

Mapa de vehículos (Marca-Modelo) como “values” y sus matrículas como “keys”

```
public static void main(String[] args) {
  Map <String,Vehicle> mapa = new TreeMap< >();
  mapa.put(“111A”, new Vehicle(”Opel”,”Crossland"));
  mapa.put(“333C”, new Vehicle(“Peugeot”,”3008"));
  mapa.put(“222B”, new Vehicle(“Citroen”,”C3"));
   System.out.println(mapa);

 Vehicle v = mapa.get(“111A”));
   System.out.println(“Obtenido vehículo: “+v));
   mapa.remove(“222B”);
   if (mapa.containsKey(“333C”))
         System.out.println("Encontrado");
   if (mapa.containsValue(v)) // Búsqueda de valor
          System.out.println("Encontrado");

    Set<String> claves = mapa.keySet(); //Obtener conjunto de claves
    System.out.println(claves);
    Collection<Vehicle> valores = mapa.values(); //Obtener colección valores
    System.out.println(valores);

   //Obtener un conjunto de pares
   Set<Map.Entry<String, Vehicle>> pares = mapa.entrySet();
   for (Map.Entry<String, Vehicle> p : pares){ //Recorrer los pares
         System.out.print(p.getKey() + “>>" + p.getValue() + “<< - ");
   }
}
```

```
{111A=Opel-Crossland, 222B=Citroen-C3, 333C=Peugeot-3008}
Obtenido vehículo: Opel-Crossland
Encontrado
Encontrado
```

Maps

Mapa de vehículos (Marca-Modelo) como “values” y sus matrículas como “keys”

```
public static void main(String[] args) {
  Map <String,Vehicle> mapa = new TreeMap< >();
  mapa.put(“111A”, new Vehicle(”Opel”,”Crossland"));
  mapa.put(“333C”, new Vehicle(“Peugeot”,”3008"));
  mapa.put(“222B”, new Vehicle(“Citroen”,”C3"));
   System.out.println(mapa);

 Vehicle v = mapa.get(“111A”));
   System.out.println(“Obtenido vehículo: “+v));
   mapa.remove(“222B”);
   if (mapa.containsKey(“333C”))
         System.out.println("Encontrado");
   if (mapa.containsValue(v)) // Búsqueda de valor
          System.out.println("Encontrado");

    Set<String> claves = mapa.keySet(); //Obtener conjunto de claves
    System.out.println(claves);
    Collection<Vehicle> valores = mapa.values(); //Obtener colección valores
    System.out.println(valores);

   //Obtener un conjunto de pares
   Set<Map.Entry<String, Vehicle>> pares = mapa.entrySet();
   for (Map.Entry<String, Vehicle> p : pares){ //Recorrer los pares
         System.out.print(p.getKey() + “>>" + p.getValue() + “<< - ");
   }
}
```

```
{111A=Opel-Crossland, 222B=Citroen-C3, 333C=Peugeot-3008}
Obtenido vehículo: Opel-Crossland
Encontrado
Encontrado
[111A, 333C]
```

Maps

Mapa de vehículos (Marca-Modelo) como “values” y sus matrículas como “keys”

```
public static void main(String[] args) {
  Map <String,Vehicle> mapa = new TreeMap< >();
  mapa.put(“111A”, new Vehicle(”Opel”,”Crossland"));
  mapa.put(“333C”, new Vehicle(“Peugeot”,”3008"));
  mapa.put(“222B”, new Vehicle(“Citroen”,”C3"));
   System.out.println(mapa);

 Vehicle v = mapa.get(“111A”));
   System.out.println(“Obtenido vehículo: “+v));
   mapa.remove(“222B”);
   if (mapa.containsKey(“333C”))
         System.out.println("Encontrado");
   if (mapa.containsValue(v)) // Búsqueda de valor
          System.out.println("Encontrado");

    Set<String> claves = mapa.keySet(); //Obtener conjunto de claves
    System.out.println(claves);
    Collection<Vehicle> valores = mapa.values(); //Obtener colección valores
    System.out.println(valores);

   //Obtener un conjunto de pares
   Set<Map.Entry<String, Vehicle>> pares = mapa.entrySet();
   for (Map.Entry<String, Vehicle> p : pares){ //Recorrer los pares
         System.out.print(p.getKey() + “>>" + p.getValue() + “<< - ");
   }
}
```

```
{111A=Opel-Crossland, 222B=Citroen-C3, 333C=Peugeot-3008}
Obtenido vehículo: Opel-Crossland
Encontrado
Encontrado
[111A, 333C]
[Opel-Crossland, Peugeot-3008]
```

Maps

Mapa de vehículos (Marca-Modelo) como “values” y sus matrículas como “keys”

```
public static void main(String[] args) {
  Map <String,Vehicle> mapa = new TreeMap< >();
  mapa.put(“111A”, new Vehicle(”Opel”,”Crossland"));
  mapa.put(“333C”, new Vehicle(“Peugeot”,”3008"));
  mapa.put(“222B”, new Vehicle(“Citroen”,”C3"));
   System.out.println(mapa);

 Vehicle v = mapa.get(“111A”));
   System.out.println(“Obtenido vehículo: “+v));
   mapa.remove(“222B”);
   if (mapa.containsKey(“333C”))
         System.out.println("Encontrado");
   if (mapa.containsValue(v)) // Búsqueda de valor
          System.out.println("Encontrado");

    Set<String> claves = mapa.keySet(); //Obtener conjunto de claves
    System.out.println(claves);
    Collection<Vehicle> valores = mapa.values(); //Obtener colección valores
    System.out.println(valores);

   //Obtener un conjunto de pares
   Set<Map.Entry<String, Vehicle>> pares = mapa.entrySet();
   for (Map.Entry<String, Vehicle> p : pares){ //Recorrer los pares
         System.out.print(p.getKey() + “>>" + p.getValue() + “<< - ");
   }
}
```

```
{111A=Opel-Crossland, 222B=Citroen-C3, 333C=Peugeot-3008}
Obtenido vehículo: Opel-Crossland
Encontrado
Encontrado
[111A, 333C]
[Opel-Crossland, Peugeot-3008]
111A>>Opel-Crossland<< - 333C>>Peugeot-3008<< -
```

Resumen sobre agrupamiento de objetos

- Las colecciones son casi siempre "mejores" que los Arrays
- La “limpieza” de elementos se hace implícita (y correctamente)
- Las colecciones sólo almacenan datos de tipo objeto
  - Para almacenar datos primitivos se usan automáticamente clases “Wrapper” (ver apartado siguiente)

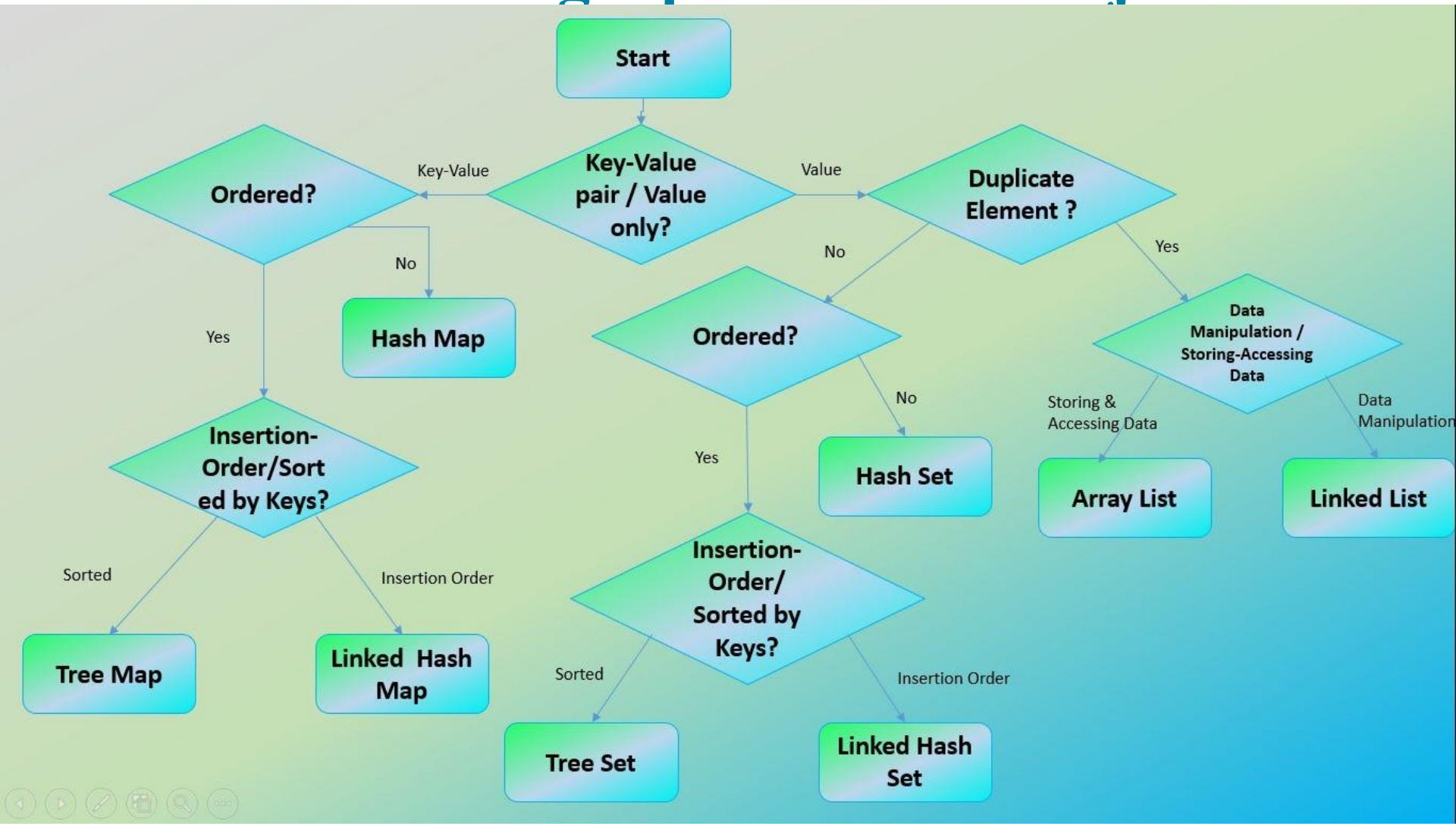
Pearson

Resumen sobre agrupamiento de objetos

Tipo	Orden Inserción	Elementos Duplicados	Orden	Elementos nulos
List – generalmente para acceso aleatorio o acceso continuo o claves numéricas				
ArrayList	Sí	Sí	No	Sí
LinkedList	Sí	Sí	No	Sí
Set – generalmente para evitar duplicados (o eliminarlos implícitamente)				
HashSet	No	No	No	Sí
LinkedHashSet	Sí	No	No	Sí
TreeSet	No	No	Sí	No
Map – para devolver valores asociados a clavese (pares key-value)				
HashMap	No	Clave única pero valores duplicados	No	1 clave Null pero muchos valores Null
LinkedHashMap	Sí	Clave única pero valores duplicados	No	1 clave Null pero muchos valores Null
TreeMap	No	Clave única pero valores duplicados	Sí	**No** permite clave Null pero sí valores Null

Pearson

Resumen sobre agrupamiento de objetos



Fuente original aquí

Pearson

Mas información en:

- JavaAPI

  https://docs.oracle.com/en/java/javase/20/docs/api/index.html
- ArrayList

  https://docs.oracle.com/en/java/javase/20/docs/api/java.base/java/util/ArrayList.html
- LinkedList

  https://docs.oracle.com/en/java/javase/20/docs/api/java.base/java/util/LinkedList.html
- HashSet

  https://docs.oracle.com/en/java/javase/20/docs/api/java.base/java/util/HashSet.html
- LinkedHashSet

  https://docs.oracle.com/en/java/javase/20/docs/api/java.base/java/util/LinkedHashSet.html
- TreeSet

  https://docs.oracle.com/en/java/javase/20/docs/api/java.base/java/util/TreeSet.html
- HashMap

  https://docs.oracle.com/en/java/javase/20/docs/api/java.base/java/util/HashMap.html
- LinkedHashMap

  https://docs.oracle.com/en/java/javase/20/docs/api/java.base/java/util/LinkedHashMap.html
- TreeMap

  https://docs.oracle.com/en/java/javase/20/docs/api/java.base/java/util/TreeMap.html

Algoritmos

- Existe una serie de algoritmos ya implementados que permiten realizar distintas operaciones sobre los datos almacenados en un contenedor:
  - ordenar, desordenar, mezclar, generar datos, **etc.**
- Son **independientes** del tipo de datos que haya almacenado en el contenedor y, en muchos casos, del tipo de contenedor.
- Existen dos clases que tienen implementados estos algoritmos:

Arrays Collections

Algoritmos

Arrays

• Implementa algoritmos genéricos sobre matrices o arrays:

- **asList**(), **binarysearch**(), **copyOf**(), **fill**(), **equals**(), **sort**(),
- https://docs.oracle.com/javase/7/docs/api/java/util/Arrays.html

Collections

• Implementa algoritmos genéricos sobre Colecciones o contenedores (algunos sobre List y otros sobre Collection):

- **addAll**(), **binarySearch**(), **copy**(), **disjoint**(), **enumeration**(), **frequency**(), **indexOfSubList**(), **max**(), **min**(), **nCopies**(), **replaceAll**(), **reverse**(), **shuffle**(), **sort**(), …

https://docs.oracle.com/javase/7/docs/api/java/util/Collections.html

Principales conceptos

- Collections (*List*, *Set* y *Map*).
- Clases wrapper (envoltorio).
- Comportamiento aleatorio (*Random*)
- Variables y métodos de clase (static). Constantes (final)
- Tipo enumerado (*Enum*)
- Definición y uso del método ***main***

Clases *Wrapper*

Colecciones y tipos primitivos

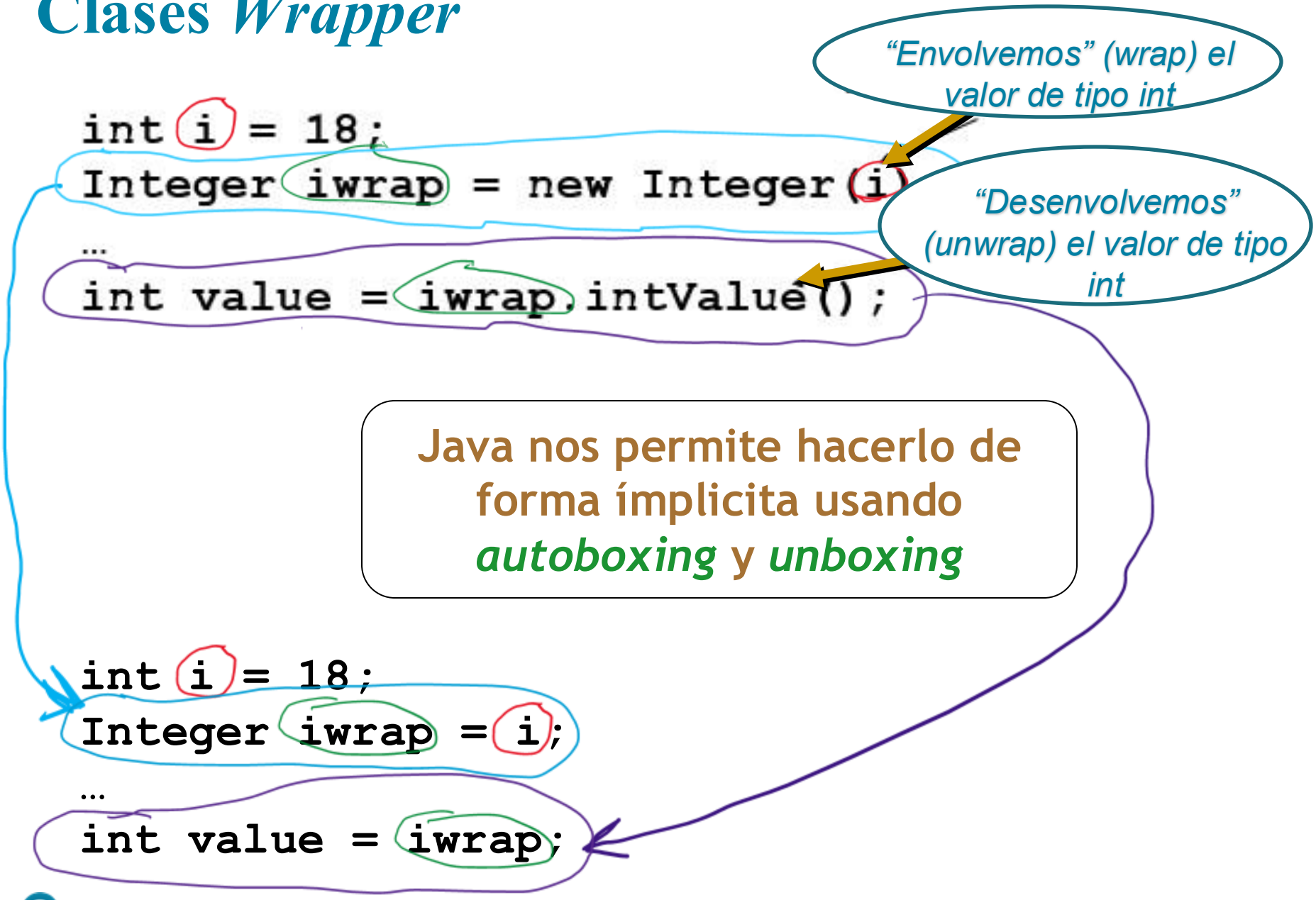
- Las clases de Collection se pueden usar con todos los tipos de clases ...
- … pero ¿qué ocurre con los **tipos primitivos: `int`, `boolean`,** etc.?
- ¿Que ocurre si queremos definir un `ArrayList` de `int`? ¿podemos hacerlo?

Clases *Wrapper*

- Los tipos primitivos no son de tipo objeto.
  - Entre otras cosas no podemos llamar a métodos con **tipos primitivos**.
  - Los valores de tipos primitivos deben ser envueltos (*wrapped*) en objetos para poder almacenarse en una colección!
- Existen clases envoltorio (*wrapper*) para todos los tipos primitivos:

Tipo primitivo	Clase *Wrapper*
int	**Integer**
float	**Float**
char	**Character**
...	...

Clases *Wrapper*



Clases *Wrapper*

Autoboxing y unboxing

El *autoboxing* y *unboxing* se realiza de forma automática cuando un valor de tipo primitivo es usado en un contexto donde se require un tipo wrapper

```
private ArrayList<Integer> markList;
...
public void storeMark(int mark)
{
    markList.add(mark);
}
```

autoboxing

```
int firstMark = markList.remove(0);
```

unboxing

https://docs.oracle.com/javase/tutorial/java/data/autoboxing.html

Clases *Wrapper*

"Parseo"

- Métodos específicos para cada tipo de datos que permiten su transformación cuando sus valores son compatibles.
  - Java ofrece métodos predefinidos de "parseo" para todos los tipos de datos primitivos.
  - "Parseo" no es "casting"*.

```
1  double d = Double.parseDouble("1.234");
2  int i = Integer.parseInt("1234");
3  boolean b = Boolean.parseBoolean("true");
4
5  double d = Double.parseDouble("Hello");
```

¡¡Excepción *NumberFormat!!*

Principales conceptos

- Collections (*List*, *Set* y *Map*).
- Clases wrapper (envoltorio).
- Comportamiento aleatorio (*Random*)
- Variables y métodos de clase (static). Constantes (final)
- Tipo enumerado (*Enum*)
- Definición y uso del método ***main***

Comportamiento aleatorio (*Random*)

- La clase de biblioteca **`Random`** se puede utilizar para generar números pseudo-aleatorios:

```
import java.util.Random;
...
Random rand = new Random();
...
int num = rand.nextInt();

int value = 1 + rand.nextInt(100);



int index = rand.nextInt(list.size());
```

Random se importa del paquete java.util

Creamos una instancia de la clase Random y la asignamos a una variable local

nextInt() devuelve un valor entero aleatorio

nextInt(100) devuelve un valor entero entre 0 y 99

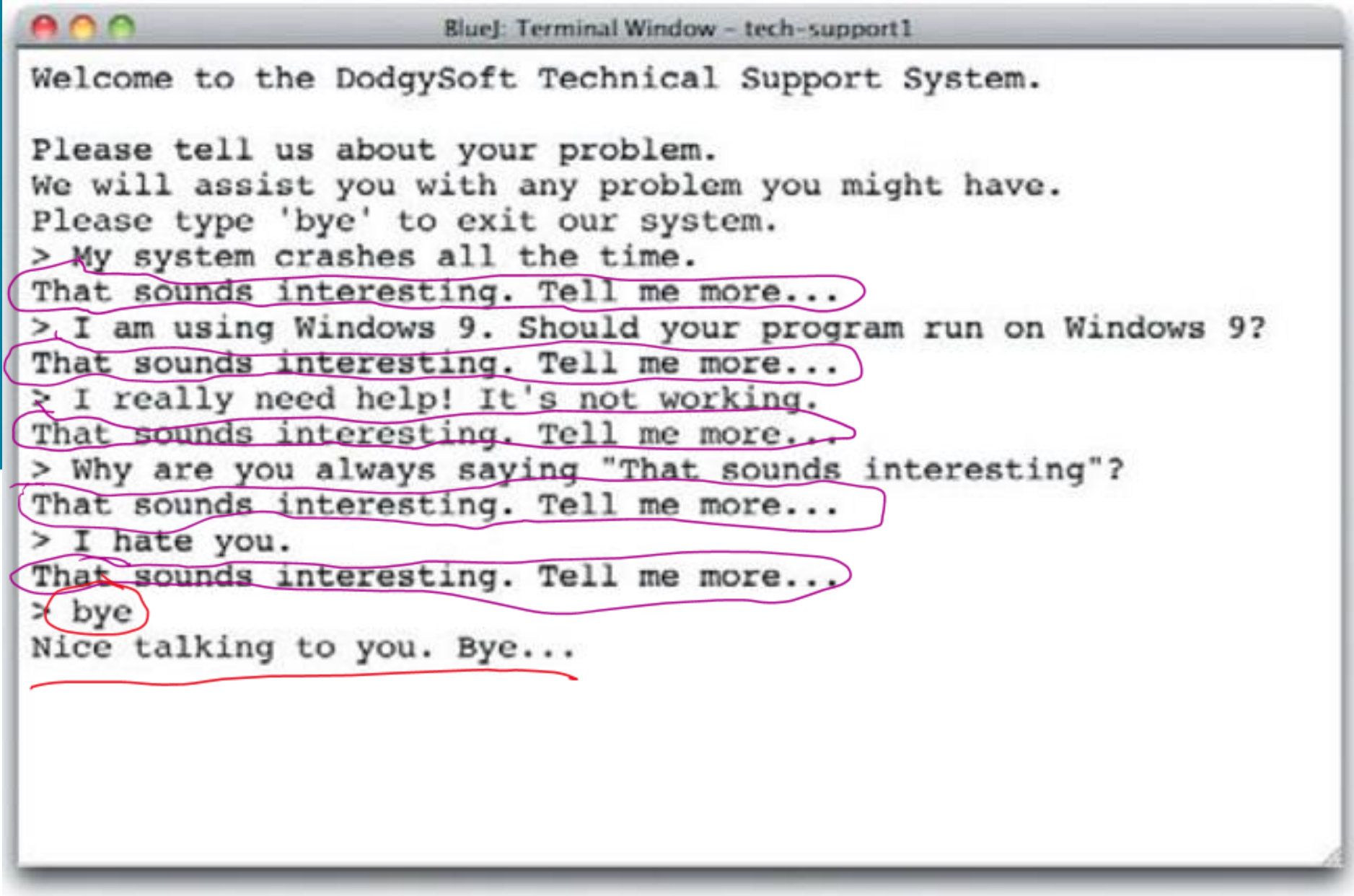
nextInt(list.size()) devuelve un valor entero entre 0 y size-1

Demo de Proyecto “tech-support-1” (Proyectos de ejemplo de Unidad 6)

Un sistema textual, de diálogo interactivo.

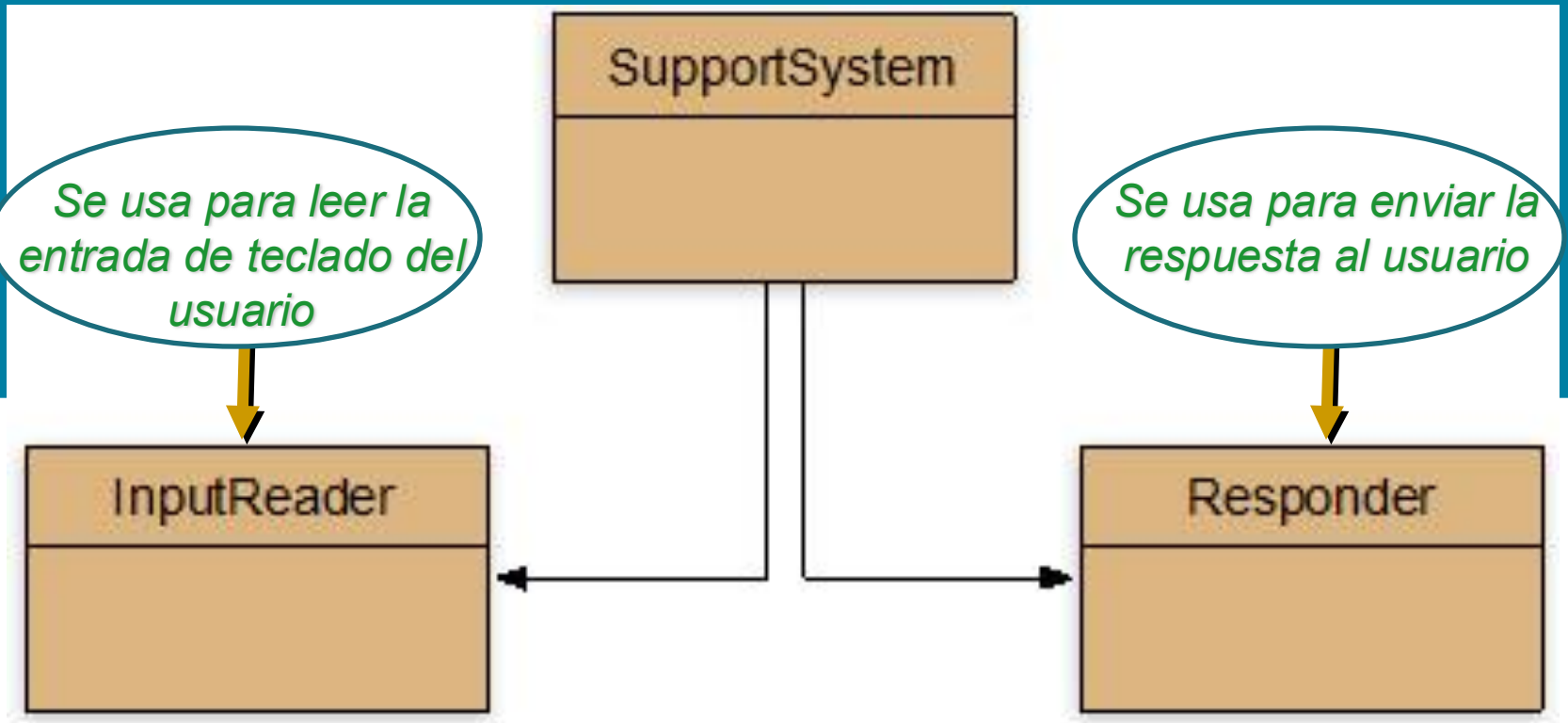
- Idea basada en ‘Eliza’ de Joseph Weizenbaum (MIT,1960s).
- Esta versión es bastante tosca pero la iremos mejorando a lo largo de esta unidad.

Demo de Proyecto “tech-support-1”



Demo de Proyecto “tech-support-1”

Modularización



Demo de Proyecto “tech-support-1”

```
public class SupportSystem
{
    private InputReader reader;
    private Responder responder;
    /**
     * Creates a technical support system.
     */
    public SupportSystem()
    {
        reader = new InputReader();
        responder = new Responder();
    }

    ...
```

Demo de Proyecto “tech-support-1”

Estructura básica.

- El sistema de soporte técnico tiene una interfaz textual
- El usuario introduce una pregunta al soporte técnico
- El programa genera una respuesta

Demo de Proyecto “tech-support-1”

Estructura del bucle principal

Método Start en clase SupportSystem

Patrón común de iteración

```
boolean finished = false;
printWelcome ();
while (!finished) {

    ...  do something

    if (we want to finish) {
        finished = true;
    }
    else {
        ...  do something more
    }
}

printGoodbye();
```

Demo de Proyecto “tech-support-1”

Estructura del bucle principal

Método Start en clase SupportSystem

Patrón común de iteración

```
boolean finished = false;
printWelcome ();
while (!finished) {
    String input = reader.getInput ();

    if (we want to finish) {
        finished = true;
    }
    else {
        ...  do something more
    }
}

printGoodbye();
```

Devuelve la línea o texto introducido por el cliente

```
private InputReader reader;
```

Demo de Proyecto “tech-support-1”

Estructura del bucle principal

Método Start en clase SupportSystem

Patrón común de iteración

```
boolean finished = false;
printWelcome ();
while (!finished) {

    String input = reader.getInput ();


    if (input.startsWith ("bye")) {
        finished = true;
    }
    else {
        ...  do something more     }
}

printGoodbye();
```

Condición de salida.

En este caso terminamos si el usuario teclea “bye”

Demo de Proyecto "tech-support-1"

Estructura del bucle principal

Método Start en clase SupportSystem

Patrón común de iteración

```
boolean finished = false;
printWelcome ();
while (!finished) {

    String input = reader.getInput ();


    if (input.startsWith ("bye")) {
        finished = true;
    }
    else {
        ...  do something more     }
}

printGoodbye();
```

¿De dónde viene '**startsWith**'?
¿Qué es? ¿Qué hace?
¿Cómo puedo obtener más información

Condición de salida.

En este caso terminamos si el usuario teclea "bye"

- Es un método invocado en '**input**' …
- Por tanto, es un método de '**String**' que podemos buscar en la documentación…

Demo de Proyecto "tech-support-1"

Estructura del bucle principal

Método Start en clase SupportSystem

Patrón común de iteración

```
boolean finished = false;
printWelcome ();
while (!finished) {

    String input = reader.getInput ();


    if (input.startsWith ("bye")) {
        finished = true;
    }
    else {
        String response = responder.generateResponse ();
        System.out.println (response);
    }
}

printGoodbye();
```

Devuelve la respuesta al cliente si el cliente introdujo cualquier otra cosa

```
private Responder responder;
```

Demo de Proyecto “tech-support-1”

Estructura del bucle principal

Método Start en clase SupportSystem

```
public void start()
{
        boolean finished = false;
        printWelcome ();
        while (!finished) {

    String input = reader.getInput ();

    if (input.startsWith ("bye")) {
         finished = true;
    }
    else {
        String response = responder.generateResponse ();
        System.out.println (response);
    }
}
printGoodbye();}
```

input es ignorado por *responder* en esta versión

Mejora para Proyecto “tech-support-1”

Rellenando un ArrayList de respuestas en Clase Responder

```
private List<String> responses;

public Responder()
{
  randomGenerator = new Random();
  responses = new ArrayList<String>();
  fillResponses();
}
public void fillResponses()
{
  responses.add("That sounds odd. Could you describe \n" + "that problem in more detail?");
  responses.add("No other customer has ever \n" + "complained about this before. \n" +
                "What is your system configuration?”);
  responses.add("That’s a known problem with Vista." + "Windows 7 is much better.");
  responses.add(”I need a bit more information on that.");
  responses.add("Have you checked that you do not \n" + "have a dll conflict?");
  responses.add("That is explained in the manual. \n" + "Have you read the manual?");
  responses.add("Your description is a bit \n" + "wishy-washy. Have you got an expert \n" +
                "there with you who could describe \n" + "this more precisely?”);
  responses.add("That’s not a bug, it’s a feature!”);
  responses.add("Could you elaborate on that?”);
}
```

Mejora para Proyecto “tech-support-1”

Seleccionando respuestas aleatorias en Clase Responder

```
private Random randomGenerator;
private List<String> responses;
public Responder()
{
    randomGenerator = new Random();
    responses = new ArrayList<String>();
    fillResponses();
}

private void fillResponses()
{
    …fill ArrayList responses with a selection of response
strings…
}

public String generateResponse()
{
    int index = randomGenerator.nextInt(responses.size());
    return responses.get(index);
}
```

devuelve un entero aleatorio en el rango legal para acceder a posiciones de responses.size()

Mejora para Proyecto “tech-support-1”

“Tokenizando” Strings en Clase InputReader

```
public Set<String> getInput()
{
    System.out.print("> ");
    String inputLine =
        reader.nextLine().trim().toLowerCase();

    String[ ] wordArray = inputLine.split(" ");
    Set<String> words = new HashSet<String>();

    for(String word : wordArray) {
        words.add(word);
    }
    return words;
}
```

Leemos de teclado, quitamos espacios al inicio y al final y convertimos a minúsculas

Dividimos las entrada filtrada en las palabras que lo componen

No almacenamos palabras repetidas

Devuelve el conjunto de palabras distintas introducidas por el usuario

Mejora para Proyecto “tech-support-1”

Devolvemos respuesta asociada a palabra clave del usuario

En Responder:

```
private Map <String, String> responseMap;
responseMap = new HashMap<String, String>();
```

← Declaración y creación

el método .put inserta una entrada

```
responseMap.put("slow",
   "I think this has to do with your hardware. \n" +
   "Upgrading your processor should solve all performance
   problems. \n" + "Have you got a problem with our
   software?");
responseMap.put("bug",
   "Well, you know, all software has some bugs. \n" + "But
   our software engineers are working very hard to fix
   them. \n" + "Can you describe the problem a bit
   further?");
responseMap.put("expensive",
   "The cost of our product is quite competitive. \n" +
   "Have you looked around and really compared our
   features?");
```

Mejora para Proyecto “tech-support-1”

Devolvemos respuesta asociada a palabra clave del usuario

En Responder:

```
public String generateResponse(Set<String> words)
{
    Iterator<String> itWords = words.iterator();
    while (itWords.hasNext()){
        String key = itWords.next();
        String response = responseMap.get(key);
        if (response != null) {
            return response;
        }
    }
    return pickDefaultResponse();
}
```

enviamos la clave

Si esa key existe en el Mapa, este devuelve el value asociado a ella – en otro caso devuelve null

Recuerda la version anterior usando un ArrayList de String:

```
public String generateResponse()
{
   int index = randomGenerator.nextInt(responses.size());
   return responses.get(index);
}
```

Respuesta aleatoria (random)

Principales conceptos

- Collections (*List*, *Set* y *Map*).
- Clases wrapper (envoltorio).
- Comportamiento aleatorio (*Random*)
- Variables y métodos de clase (static). Constantes (final)
- Tipo enumerado (*Enum*)
- Definición y uso del método ***main***

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

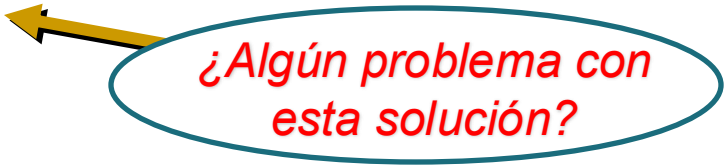
Imagina escenario relacionado con vehículos

Queremos convertir una velocidad desde:

kilómetros/hora -> millas/hora

Posible solución (v1):

```
1  public class Vehicle{
2
3     public double speedInMph(double speedInKmh){
4        return speedInKmh * 0.621371192;
5     }
6  }
```



- ***Ejemplo de Uso:***

```
Vehicle myVehicle = new Vehicle();
System.out.println(myVehicle.speedInMph(12.5));
```

Imagina escenario relacionado con vehículos

Problemas de la solución (v1):

- El factor de conversión se conoce cuando escribimos el código
- Cuando revisamos código dos semanas después, parece algo arbitrario

Posible solución (v2):

- podemos escribir un comentario
- **mejor: asignarlo a una variable con un identificador significativo**

```
1   public class Vehicle{
2     private double conversionFactor= 0.621371192;
3     public double speedInMph(double speedInKmh){
4        return speedInKmh * conversionFactor;
5     }
6   }
```

- ***Ejemplo de Uso:***

```
Vehicle myVehicle = new Vehicle();
System.out.println(myVehicle.speedInMph(12.5));
```

¿Realmente necesitamos un objeto de tipo Vehicle?

Variables de clase

- Una variable de clase es compartida por **todas** las instancias de la clase.
- De hecho, pertenece a la clase y existe independientemente de las instancias.
- Se define con la palabra clave `static`.
  - private **static** double GRAVITY = 9.8;
- El acceso a las variables static públicas se hace por el nombre de la clase (NO por el nombre del objecto), ej.:
  - `BouncingBall.GRAVITY`



Nombre de la clase

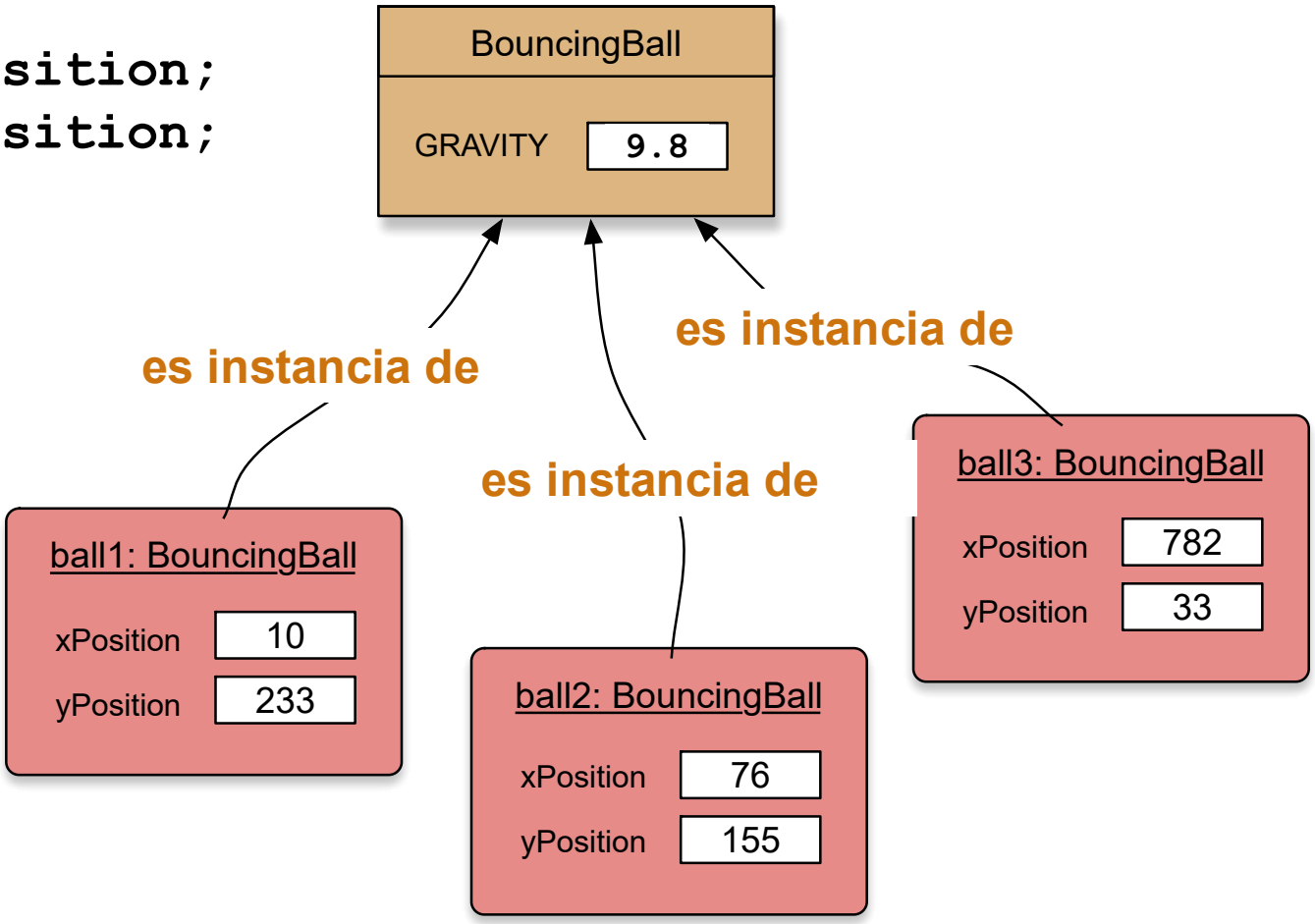
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Variables de clase

```
private static double GRAVITY = 9.8;

private int xPosition;
private int yPosition;
```



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Variables de clase

```
1  public class Car{
2    private static String color = "yellow";
3      [...]
4  }
5
6  public static void main(String[] args){
7     Car car1 = new Car();
8     Car car2 = new Car();
9
10 }
```



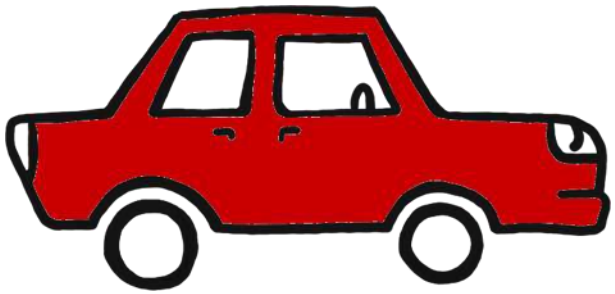


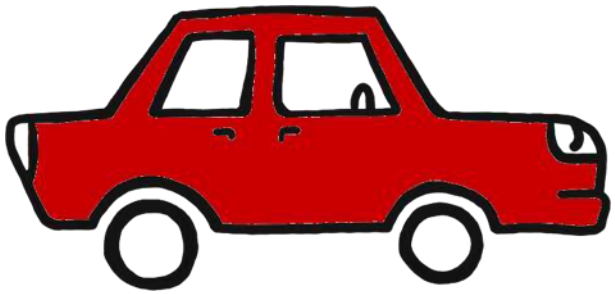
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Variables de clase

```
1  public class Car{
2    private static String color = "yellow";
3      [...]
4  }
5
6  public static void main(String[] args){
7     Car car1 = new Car();
8     Car car2 = new Car();
9     car2.setColor("red");
10    }
```





Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Constantes

- Son variables que, una vez definidas, su valor permanece inalterable.
- Se definen con la palabra clave **final**.
  - `private final int SIZE = 10;`
- Los **campos final** pueden inicializarse en su declaración o en el constructor.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Constantes de clase

- Combinación de campos con **`static`** y **`final`**

- **`static`**: especifica que todos los objetos de esta clase comparten este campo – pertenece a la clase
- **`final`**: especifica que el valor dado no puede ser cambiado – es una constante
  - **`final`** puede aplicarse a cualquier campo (clase o datos), métodos, variables locales o parámetros.
- Se suelen poner en mayúsculas

```
public static final int BOILING_POINT = 100;
```

- Las constantes deberían tener visibilidad **public**
  - todo el mundo se beneficia de la constante
  - su valor no se puede modificar

Métodos de clase

- Hasta ahora, hemos usado solo *métodos de instancia*
  - que se invocan en una instancia (objeto) de una clase
- Sin embargo, *los métodos de clase* son distintos
  - pueden invocarse SIN un objeto de una clase
- Como ocurría en las variables de clase, los métodos de clase PERTENECEN a la clase no a las instancias
  - el nombre de la clase es suficiente para invocarlos

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

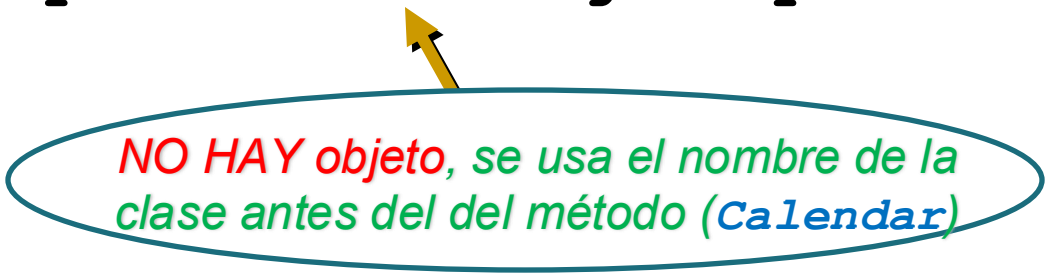
Métodos de clase

- Se definen usando el modificador **`static`**

```
public static int getDaysThisMonth()
```

- Los métodos static se invocan usando su nombre de clase:

```
int days = Calendar.getDaysThisMonth();
```



- Aunque también puede invocarte desde una instacia existente con el mismo resultado:

```
Calendar myCalendar = new Calendar();

int days = myCalendar.getDaysThisMonth();
```

Limitaciones de los métodos de clase

- Un método de clase (static)
  - se puede invocar independientemente de cualquier instancia
  - se puede invocar sin necesidad de tener ninguna instancia creada.
- Por tanto:
  - NO pueden acceder a campos de instancias dentro de su propia clase.
  - NO pueden llamar a métodos de instancias dentro de su propia clase.



Sintaxis de un método de clase:

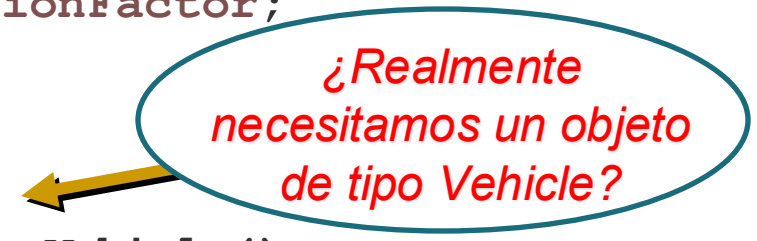
```
public static <<tipoRetorno>> <<nombreMetodo>>(...){
  [...] // aquí sólo variables locales o estáticas
        // también: sólo llamadas a otros métodos estáticos
}
```

¿Recuerdas escenario de vehículos?

Problemas de la solución (v2):

- ¿Realmente necesitamos un objeto de tipo Vehicle para poder hacer la conversión de kilómetros/hora a millas/hora?

```
1  public class Vehicle{
2     private double conversionFactor= 0.621371192;
3     public double speedInMph(double speedInKmh){
4        return speedInKmh * conversionFactor;
5     }
6  }
```



- ***Ejemplo de Uso:***

```
Vehicle myVehicle = new Vehicle();
System.out.println(myVehicle.speedInMph(12.5));
```

¿Recuerdas escenario de vehículos?

Problemas de la solución (v2):

- ¿Realmente necesitamos un objeto de tipo Vehicle para poder hacer la conversión de kilómetros/hora a millas/hora?

Posible solución (v3):

- **mejor: uso de constante de clase y método de clase**

```
1  public class Vehicle{
2     public static final double conversionFactor = 0.621371192;
3     public static double speedInMph(double speedInKmh){
4        return speedInKmh * conversionFactor;
5     }
6  }
```

- ***Ejemplo de Uso:***

```
System.out.println(Vehicle.speedInMph(12.5));
```

Invocamos método de clase sin crear objeto de tipo Vehicle

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Principales conceptos

- Collections (*List*, *Set* y *Map*).
- Clases wrapper (envoltorio).
- Comportamiento aleatorio (*Random*)
- Variables y métodos de clase (static). Constantes (final)
- Tipo enumerado (*Enum*)
- Definición y uso del método ***main***

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Tipos enumerados

- Es una característica del lenguaje.
- Utiliza **`enum`** en lugar de **`class`** para introducir un **nombre de tipo**.
- Su uso más simple es definir un conjunto de nombres significativos.
  - Es una alternativa a las constantes static **`int`**.
  - Si los valores de las constantes fueran arbitrarios.
- Las definiciones enum también pueden tener campos, constructores y métodos

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Tipos enumerados

```
CircuitoPrueba portugal = new CircuitoPrueba("Portugal", Distancia.INTERMEDIA);
```

```
public enum Distancia
{
    CORTA ("CORTA",250.0),
    INTERMEDIA ("INTERMEDIA",275.0),
    LARGA ("LARGA",300.0);

    private final String nombre;
    private final double valor;

    Distancia(String nombre, double valor) {
        this.nombre = nombre;
        this.valor = valor;
    }
    public String getNombre() {
        return nombre;
    }
    public double getValor() {
        return valor;
    }
    @Override
    public String toString() {
        return "<dist: " + getNombre() + " (valor: "+getValor()+")>";
    }
```

```
public class CircuitoPrueba
{
    private Distancia distancia;
    private String nombre;

    protected CircuitoPrueba(String nuevoNombre, Distancia distancia){
        nombre=nuevoNombre;
        setDistancia(distancia);
    }

    public String getNombre(){
        return nombre;
    }

    public Distancia getDistancia(){
        return distancia;
    }

    public double getValorDistancia(){
        return distancia.getValor();
    }

    public void setDistancia(Distancia distancia){
        this.distancia=distancia;
    }

    public String toString() {
        return "<circuito:" + getNombre() + "> "+ getDistancia();
    }
}
```

```
<circuito:Portugal> <dist: INTERMEDIA (valor: 275.0)>
```

- Vamos a ver más ejemplos de uso de tipos enumerados en la unidad "*Mejorando la calidad del código*"

Principales conceptos

- Collections (*List*, *Set* y *Map*).
- Clases wrapper (envoltorio).
- Comportamiento aleatorio (*Random*)
- Variables y métodos de clase (static). Constantes (final)
- Tipo enumerado (*Enum*)
- Definición y uso del método ***main***

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

El método main

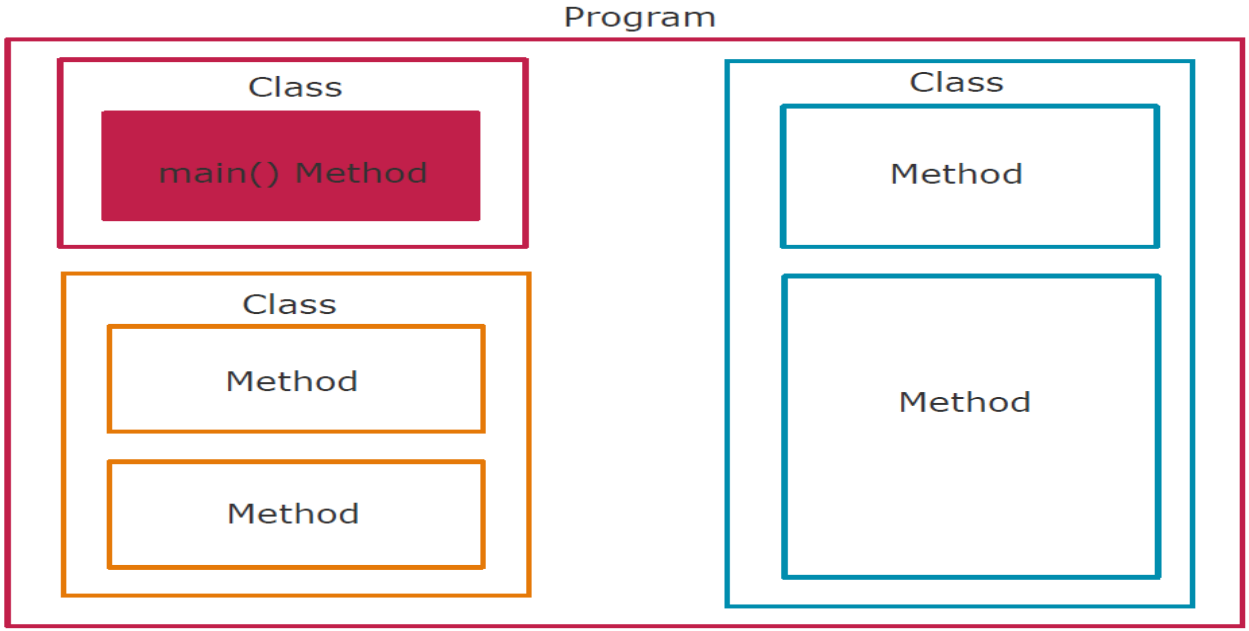
BlueJ nos ha abstraído de esto hasta ahora

- Punto de inicio de cualquier programa en Java:

```
public static void main(String[] args)
```

- Solo puede estar definido dentro de una de las clases que compongan el programa





Otra bibliografía:

- Tony Gauvin - Introduction to Programming. Presentations. University of Maine. Fort Kent.
- Dr. Becka Morgan- Introduction to Computer Science. Presentations. University of Western Oregon.
- Christiane Hagedorn, Ann Katrin Kuessner, Thomas Staubitz, Ralf Teusner. Object-Oriented Programming in Java. OpenSAP Courses.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved